

TECHNION — ISRAEL INSTITUTE OF TECHNOLOGY
THE ANDREW AND ERNA VITERBI FACULTY OF ELECTRICAL & COMPUTER
ENGINEERING
VLSI Laboratory — Final Project

Design Controller for an Artificial Neural Network (ANN) Accelerator and its Peripherals

Submitted by

Mohammad Abu Ammar
Lana Bakrieh

Supervisor: *Nir Ben-Haim*

Laboratory: *VLSI Lab*

Semester: *Winter 2025/2026*

July 20, 2026

Contents

1	Introduction	1
1.1	Motivation: The Memory Wall and Compute in Memory	1
1.2	Project Context and Scope	1
1.3	Contributions	1
1.4	Report Organization	2
2	Background	3
2.1	Compute in Memory and the MVM Kernel	3
2.2	The Y-Flash Floating-Gate Memristor	4
2.3	The Write–Verify Problem	5
3	System Architecture	6
3.1	Top-Level Organization	6
3.2	Host-to-Core Transaction Word	7
3.3	Command Set	8
3.4	Module Partitioning Philosophy	8
4	RTL Module Design	10
4.1	Parallel Interface	10
4.2	ANN Controller	12
4.2.1	Registered Capture of Host Data	13
4.2.2	Address Round-Trip and Payload Multiplexing	13
4.2.3	Main FSM: Hardware Block Flow	13
4.2.4	Sub-FSMs and the Closed-Loop Recovery Policy	16
4.2.5	Pulse Generation and LUT-Based Programming	20
4.2.6	Inference Path	22
4.2.7	Recovery Loop, End to End	23
5	Verification Methodology	25
5.1	Verification Objectives	25
5.2	The Behavioral Mock: Standing in for the Analog ANN	25
5.3	Forcing the Recovery Branches: Fault Injection	26
5.4	End-of-Run Checks	27
5.5	Confidence Argument for Analog Integration	27
6	Results	28
6.1	Regression Coverage and Outcomes	28
6.2	Static Contract: Address and Pulse-Mode Packing	29
6.3	Dynamic Protocol: All Modules Integration	29
6.4	Closed-Loop Recovery	30
6.5	Inference Dataflow	31
6.6	What the Results Establish and What They Do Not	33
7	Limitations and Future Work	34
7.1	Limitations and Technical Debt	34
7.2	Future Work - Constrained-Random Stimulus and Coverage	34
7.3	Future Work - Synthesis, Integration, and Post-Silicon Validation	35

8 Conclusion	36
Supplementary Material	37
References	38

List of Figures

2.1	$M \times N$ memristor crossbar performing an analog matrix–vector multiplication. Row read-voltages $\{V_i\}$ drive programmable conductances G_{ij} ; per-column currents $I_j = \sum_i G_{ij}V_i$ are digitized at the array periphery.	4
2.2	Physical structure of the Y-Flash floating-gate memristor. The cell pairs a longer-channel <i>read</i> transistor with a shorter-channel <i>injection</i> transistor that share a single floating gate and a common drain, and is fabricated in a standard single-poly CMOS process without additional masks. The three operations - Read at low gate bias, Program by channel-hot-electron injection onto the floating gate, and Erase by band-to-band tunneling - are selected entirely by the external bias configuration, while per-cell selection is achieved at the source terminal so that un-selected cells in the same row or column remain undisturbed.	5
3.1	Top-level block diagram. The host drives a transaction word and a command token into the combinational <i>Parallel Interface</i> , which qualifies the packet and produces a validated bus consumed by the <i>ANN Controller</i> . The controller sequences commands toward the analog ANN core over a packed core word and a pulse-mode bus, and it simultaneously drives the <i>Input Buffer</i> that stores inference pixels and the expected-weight nibble used during verify. A single status bit from the core (<i>op_done</i>) closes the loop in the program and erase sub-FSMs.	6
3.2	Pictorial layout of the 32-bit transaction word and the internal round-trip used by the controller: the host one-hot tail is decoded into a 16-bit packed binary address, the FSM captures and operates on the packed address, and the outbound core word is rebuilt by single-bit left-shifts into the one-hot domain just before reaching the core.	8
4.1	Parallel Interface I/O block diagram. The module exposes a host-side port set (<i>host_data</i> , <i>host_cmd</i>) and a controller-side port set (<i>address</i> , <i>data</i> , <i>cmd</i> , <i>valid</i>); all signals are combinational, so the entire block can be timed as a single decode stage.	10
4.2	32-bit host transaction word split. The high byte carries the payload (weight nibble or pixel byte); the remaining 24 bits are partitioned into four concatenated one-hot fields - <i>PE</i> (4 bits), <i>SA</i> (4 bits), <i>col_ID</i> (8 bits), and <i>row_ID</i> (8 bits) - each of which must be <i>strictly</i> one-hot for the interface to assert <i>valid</i>	11
4.3	Translated, packed address format passed to the controller. Each one-hot field of the host word is reduced to its binary index, and the four indices are concatenated into a 16-bit packed address: <i>PE</i> occupies bits [15:12], <i>SA</i> bits [11:8], <i>col_ID</i> bits [7:4], and <i>row_ID</i> bits [3:0]. Storage and arithmetic inside the controller use this packed form; the one-hot encoding is rebuilt by single-bit left-shifts only at the core-facing output.	11

- 4.4 ANN Controller I/O block diagram. Host-side inputs (`address`, `data`, `cmd`, `valid`) arrive pre-validated from the parallel interface; analog-side inputs (`op_done`, `weight_read_data`) close the program/erase and verify loops. The controller emits the packed core word (`addr_out`, `data_out`), the three-bit pulse-mode bus (`pulses`), the buffer-control port set, and a small bank of status flags (`erase_retry_exhausted`, `verify_failed`). Note that this diagram shows a high-level abstraction; not all signals or ports are explicitly represented. Comprehensive signal and port tables are provided in the project repository. . . . 12
- 4.5 Command decode and state routing from `S_IDLE`. While `valid` is low or `cmd` equals `CMD_HIZ`, the FSM holds in `S_IDLE`; otherwise a `unique case` on `cmd` routes into exactly one operation branch (`S_PROGRAM`, `S_ERASE`, `S_READ`, or `S_COLLECT_DATA`). This high-level view highlights the routing logic; the full nine-state transition set is detailed in the subsequent architecture sections. . . . 13
- 4.6 The 9 states and the closed-loop recovery policy. Edges out of `S_VERIFY` and `S_ERASE` summarize the decisions taken in the sub-FSMs - match, under-programmed, over-programmed, retry-exhausted - which implement the `PROG` $\rightarrow$ `VERIFY` $\rightarrow$ `ERASE` $\rightarrow$ `RE-PROG` recovery policy. 14
- 4.7 Program sub-FSM. Five sub-states map onto the physical phases of a Y-Flash programming operation: `HIZ`, `SELECT`, `WRITE`, `WAIT_ACK`, `COMPLETE`. 16
- 4.8 `CHECK` state branching logic: match, under-programmed, over-programmed. The three outcomes drive the closed-loop recovery policy - success returns to idle, under-programmed retries (or falls back to erase when the per-cell retry budget is exhausted), and over-programmed triggers an erase followed by reprogramming. 18
- 4.9 Erase sub-FSM. The state graph is symmetric with the program sub-FSM (Figure 4.7), reflecting the equivalent physical sequencing of the two device operations. The host/verify flag set on entry steers the exit from `ERASE_COMPLETE` between a return to idle (host-initiated, or verify-initiated with the retry budget exhausted) and a re-entry into `S_PROGRAM` (verify-initiated with retries available). 19
- 4.10 Physical pulse-train structure on the three-bit pulse-mode bus. A burst is the concatenation of `pulse_num` discrete active pulses, each T_{prog} cycles wide, separated by single-cycle `HIZ` (`3'b000`) idle gaps. The active mode (`3'b010` for program in the illustration) is driven only while the gating Boolean is high; between pulses, and after the burst completes, the bus reverts to `HIZ`. Total burst length is therefore $pulse_num \cdot T_{prog} + (pulse_num - 1) \cdot PULSE_GAP$. *Numeric example, LUT-driven first PROG:* for an expected weight of $w_e = 2$ the LUT emits a macro-repeat count $R = 3$ (circled in red). With the baseline configuration $T_{prog} = 2$, `pulse_num` = 1, and `PULSE_GAP` = 1, the inner macro is two cycles long; the controller plays $R = 3$ back-to-back copies of that macro separated by single-cycle `HIZ` gaps (annotated 1, 2, and 3), yielding a total `PROG_WRITE` duration of $3 \cdot 2 + 2 \cdot 1 = 8$ cycles, of which 6 are active (`PULSE_MODE_PROG` on the bus) and 2 are idle `HIZ` gaps between copies. 21
- 4.11 LUT-based macro-repeat structure for the programming attempts. (a) For the first attempt, the controller concatenates a number of macros given by the LUT entry for the expected weight, separated by idle windows. (b) The inner macro detail, showing one standard burst of discrete pulses. (c) Retry attempts on the same cell switch to a single-cycle nudge train, on the assumption that the first attempt has already moved the cell most of the way to the target. 22
- 4.12 Inference dataflow. Eight pixel bytes are streamed in collect, written into the buffer in row-major order, then exposed bit-serially across eight parallel lanes during compute. The bit-select line advances from zero to seven so the analog core receives one bit of each pixel per cycle. 23

4.13	Closed-loop recovery policy. The <code>PROG</code> $\rightarrow$ <code>VERIFY</code> $\rightarrow$ <code>ERASE</code> $\rightarrow$ <code>RE-PROG</code> loop maps every analog failure mode to a deterministic digital recovery path.	24
5.1	Verification environment. The three RTL modules are instantiated in their real host-to-core order; the analog ANN array is replaced by a behavioral mock with a shadow weight matrix, a combinational readback generator driven by the outbound core word, and a clocked <i>op_done</i> generator that follows the controller's own sub-FSM. Fault injection at the readback line during the verify window forces the under- and over-programmed branches of the recovery FSM.	26
6.1	Closed-loop recovery (Match case): The readback data equals the expected weight (2). The FSM naturally exits the <code>S_PROGRAM</code> $\rightarrow$ <code>S_VERIFY</code> loop and returns to <code>S_IDLE</code>	30
6.2	Closed-loop recovery (Under-programmed case): A readback mismatch (read 1 vs. expected 2) triggers the retry logic. The FSM immediately loops back into <code>S_PROGRAM</code> to apply the short single-cycle nudge train without returning to idle.	31
6.3	Closed-loop recovery (Over-programmed case): The FSM detects a severe overshoot (read 6 vs. expected 5) in <code>S_VERIFY</code> . This forces the controller into the penalty <code>S_ERASE</code> phase to clear the cell, before automatically returning to <code>S_PROGRAM</code> to rewrite the data.	31
6.4	Inference dataflow (Transaction overview): The waveform shows the FSM progressing linearly through <code>S_COLLECT_DATA</code> $\rightarrow$ <code>S_COMPUTE</code> $\rightarrow$ <code>S_IDLE</code> . Annotations: <i>Yellow oval:</i> FSM state transitions. <i>Yellow rectangle:</i> <code>ann_address</code> during the data collection phase. <i>Purple rectangle:</i> Asserted <code>pulses</code> bus during the compute phase. <i>Red rectangle:</i> Bit-serial outputs (D0-D7) and the <code>buf_bit_sel</code> counter incrementing from 0 to 7.	32
6.5	Inference dataflow (Collect phase detail): Eight successive data-load cycles capture the incoming pixels into the buffer registers, driven by the incrementing write address.	32
6.6	Inference dataflow (Compute phase detail): The <i>bit_sel</i> line steps from 0 to 7 across consecutive cycles. Synchronously, the eight bit-serial lanes (D0-D7) extract and present the corresponding bit of each captured pixel in parallel. . .	33

List of Tables

3.1	Fields of the host-to-core transaction word.	7
3.2	Host command set. Each command class is encoded as a 3-bit token on the host <code>cmd</code> input; the same 3-bit value is re-used at the core-facing pulse-mode bus, so command decode at the boundary and pulse-mode generation at the core share a single encoding space.	8
4.1	Top-level command decode from <code>S_IDLE</code> . Each branch is entered combinatorially on the first cycle that <code>valid</code> is high; subsequent transitions of the destination state are documented in Table 4.2.	14
4.2	Main-FSM states. The <code>PROG</code> $\rightarrow$ <code>VERIFY</code> $\rightarrow$ <code>ERASE</code> $\rightarrow$ <code>RE-PROG</code> recovery policy is implemented by the branches out of <code>S_VERIFY</code> and <code>S_ERASE</code> ; sub-FSM-internal sequencing is documented in Tables 4.3 to 4.5.	15
4.3	Program sub-FSM states. The pulse train of <code>PROG_WRITE</code> is selected by the LUT-based shaper on the first attempt and falls back to a short single-cycle nudge train on retries (Section 4.2.5).	17
4.4	Verify sub-FSM states. The four states correspond to the readback pulse-train window (<code>IDLE</code> , <code>WAIT</code>), the comparison decision (<code>CHECK</code>), and the closing handshake to the main FSM (<code>DONE</code>). Branches in <code>CHECK</code> reach back into <code>S_PROGRAM</code> or <code>S_ERASE</code> of the main FSM (Table 4.2).	18
4.5	Erase sub-FSM states. The structure mirrors Table 4.3 state-for-state; the only substantive difference is the closing branch in <code>ERASE_COMPLETE</code> , detailed in Table 4.6.	20
4.6	Exit rules from <code>ERASE_COMPLETE</code>	20
6.1	Directed regression coverage. Each row corresponds to a focused test of one architectural claim. <i>Cases</i> counts the number of distinct stimuli inside the bench.	28
6.1	Directed regression coverage (continued)	29

Chapter 1

Introduction

1.1 Motivation: The Memory Wall and Compute in Memory

The energy and latency costs of moving data between memory and arithmetic units dominate modern computing workloads, ANN inference and training foremost among them. This *memory wall* is intrinsic to any architecture that keeps storage and computation separate [1–3], and the penalty is largest for ANN workloads whose dominant kernel - matrix–vector multiplication (MVM) over large weight matrices - is data-movement-bound.

Compute in memory (CIM) confronts the bottleneck by performing the computation directly inside the memory array, exploiting both the physical properties of the device and the parallelism of array-level organization [2, 3]. Non-volatile resistive-switching devices - RRAM, PCM, and floating-gate memristors - can store a quantized synaptic weight as a programmable conductance, turning a 2D crossbar into a one-shot MVM engine [1, 4].

1.2 Project Context and Scope

This report documents our final project at Technion “**Design Controller for an Artificial Neural Network (ANN) Accelerator and its Peripherals**”. The project addresses the digital control layer that surrounds an analog CIM core: a controller that decodes host traffic, sequences program, read, erase, and inference operations, and drives an analog ANN core based on Y-Flash floating-gate memristors through a packed transaction word and a multi-bit pulse-mode bus. The target inference workload for the accelerator is image classification on the 8×8 MNIST dataset - a downsampled version of the standard benchmark of 28×28 grayscale images of handwritten digits (0–9), where each pixel is quantized to one byte. Furthermore, each synaptic weight is quantized to 4 bits, mapping to a discrete range of values (0–15). The 64 input pixels of an 8×8 image are streamed eight at a time into the input buffer and consumed by the analog core through eight bit-serial lanes; consequently, these dataset and precision parameters set the size of the per-cycle activation window and the bit-width of the controller’s collect/compute datapath.

The design is motivated by a direct consequence of the device physics: a single *write-once* operation cannot be relied upon, because device-to-device and cycle-to-cycle variability displace the resulting conductance from the target. A closed-loop *program–verify–(re-program or erase)* control policy is therefore required to drive stored weights toward their intended values [1, 3, 4]. The central piece of digital control is consequently a hierarchical FSM that mediates between the host and the analog core and that implements this recovery logic. The project scope is confined to RTL implementation and functional verification; netlist generation, timing closure, and power/area analysis are out of scope.

1.3 Contributions

The engineering contributions of this work are organized along three axes:

- **A controller-centric RTL architecture**, factored into a combinational host interface, a hierarchical FSM controller with three cooperating sub-FSMs for programming,

verification, and erase, and a synchronous input buffer that stages inference pixels and stores expected weights for the verify path. The controller implements a data-driven pulse-shaping mechanism in which the first programming attempt for each cell is derived from a look-up table (LUT) indexed by the expected weight.

- **A closed-loop recovery policy** that resolves the write–verify problem at the digital control layer. After every programming attempt the controller reads the cell back, compares the readback against the expected weight, and branches into a short retry pulse train (under-programmed), an erase followed by reprogramming (over-programmed, or after retry exhaustion), or termination on a successful verify.
- **A directed verification environment** that exercises this recovery policy without access to the physical analog core. The environment models the memristor array with a behavioral shadow weight matrix, generates an analog-side handshake that follows the controller’s own sub-FSM state, and deliberately injects under- and over-programmed readbacks during the verify window so that the recovery branches are observed in simulation. This is the methodological idea that lets the project deliver high confidence in the RTL prior to integration with the real analog ANN.

1.4 Report Organization

Chapter 2 reviews CIM, the Y-Flash device, and the write–verify problem that motivates the controller architecture. Chapter 3 presents the three-module stack, the transaction word, and the host command set. Chapter 4 is the technical heart of the report: each module is described as an architectural unit, with emphasis on the main FSM, the three sub-FSMs, and the LUT-based pulse shaper. Chapter 5 documents the verification strategy and the behavioral mock plus fault-injection methodology. Chapter 6 consolidates the simulation evidence. Chapter 7 catalogs the limitations of the present work and proposes a roadmap for follow-on verification. Chapter 8 closes the report. Comprehensive supporting material - full FSM diagrams, module port tables, addressing/packing conventions, pulse timing arithmetic, testbench mechanics, and RTL source listings - is collected in the project repository (see Supplementary Material at the end of the report).

Chapter 2

Background

This chapter establishes only the context required to justify the controller’s closed-loop architecture: the energetic case for compute in memory (CIM), the physical mechanism by which a crossbar implements a matrix–vector multiplication (MVM), the Y-Flash device that this controller drives, and the write–verify problem the device’s non-idealities impose on the surrounding digital logic.

2.1 Compute in Memory and the MVM Kernel

Modern data-centric workloads - particularly neural network inference - are bottlenecked by the energy cost of moving operands between memory and arithmetic units rather than by the arithmetic itself; this is the *memory wall* [1–3]. CIM sidesteps the bottleneck by embedding the computation *inside* the memory array. For the dominant ANN kernel, MVM over a stored weight matrix, the crossbar topology is structurally aligned with the dataflow: weights are stored at the crosspoints, input activations are applied on the rows, and the per-cell contributions sum physically on the columns.

In algebraic form the array implements $\vec{y} = W\vec{x}$ through two physical laws. **Ohm’s law** at each crosspoint produces a current $I_{ij} = G_{ij}V_i = w_{ij}x_i$, where the programmable conductance G_{ij} encodes the synaptic weight w_{ij} . **Kirchhoff’s current law** on each column sums these currents into $I_j = \sum_i w_{ij}x_i$, yielding one MAC per column per read [1–3]. Figure 2.1 sketches the canonical arrangement.

The energy advantage of this organization is intrinsic: an entire MVM row is consumed in a single access rather than as one memory cycle per weight. The benefit, however, is delivered only when the surrounding periphery - addressing, pulse sequencing, calibration, error handling [3] - preserves the array-level efficiency. Three requirements follow directly for the digital controller: every cell must be individually *addressable*, every operation must be *pulse-shaped* on the appropriate terminals, and every program must be *verifiable* against a target value. The architecture in Chapter 3 maps onto these three requirements.

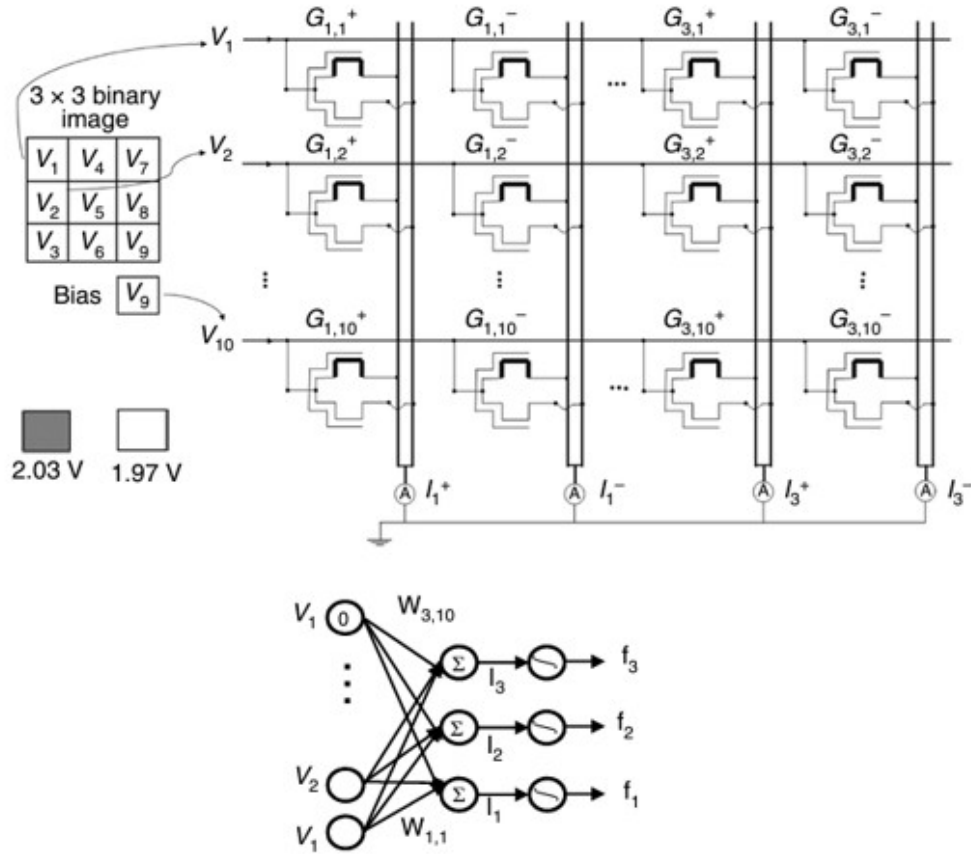


Figure 2.1: $M \times N$ memristor crossbar performing an analog matrix–vector multiplication. Row read-voltages $\{V_i\}$ drive programmable conductances G_{ij} ; per-column currents $I_j = \sum_i G_{ij}V_i$ are digitized at the array periphery.

2.2 The Y-Flash Floating-Gate Memristor

Among the candidate CIM devices - RRAM, PCM, MRAM, and floating-gate variants [1, 4] - the analog core this controller drives uses the *Y-Flash* floating-gate memristor, the device technology under development in the supervisor’s group [4].

The cell is built in a standard single-poly CMOS process without additional masks and contains two transistors - a longer-channel *read* transistor and a shorter-channel *injection* transistor - that share a common floating gate and a common drain. Three operations are exposed through distinct external bias configurations: **Read** at $V_R < 2.5$ V exposes the stored charge as a subthreshold current in the 1 nA–5 μ A range; **Program** at $V_P > 4$ V injects hot electrons onto the floating gate, raising threshold and reducing conductance; **Erase** at $V_E > 7$ V discharges the gate via band-to-band tunneling [4]. The selection is performed at the cell itself - de-biasing the source defeats programming on un-selected cells - which makes the device naturally compatible with per-cell one-hot row/column selection from the digital side. The physical arrangement of the two-transistor cell and the shared floating gate is sketched in Figure 2.2.

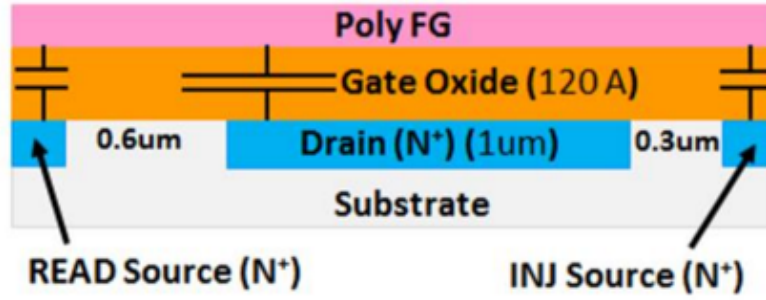


Figure 2.2: Physical structure of the Y-Flash floating-gate memristor. The cell pairs a longer-channel *read* transistor with a shorter-channel *injection* transistor that share a single floating gate and a common drain, and is fabricated in a standard single-poly CMOS process without additional masks. The three operations - Read at low gate bias, Program by channel-hot-electron injection onto the floating gate, and Erase by band-to-band tunneling - are selected entirely by the external bias configuration, while per-cell selection is achieved at the source terminal so that un-selected cells in the same row or column remain undisturbed.

2.3 The Write–Verify Problem

No single unconditional program pulse can be relied upon to place a resistive cell at its intended conductance. Sebastian et al. describe the canonical mitigation as an iterative *write–read–decide* strategy: each program pulse is interleaved with a verify read, the readback is compared against the target, and subsequent pulses are adapted (or replaced by an erase) until the cell converges or a retry budget is exhausted [1, 3, 4]. Three architectural consequences for the controller follow:

- **One-hot addressing at the boundary.** The host transaction selects exactly one cell per block through concatenated one-hot fields, mirroring the per-cell deselect mechanism the Y-Flash exposes during program and erase [4].
- **A multi-bit pulse-mode bus.** Read, program, erase, and inference each map to a distinct time-multiplexed burst encoding rather than a generic write bit-pattern, matching the device-level distinction between the three bias configurations.
- **A hierarchical FSM with explicit recovery branches.** The controller must implement a program–verify–(re-program or erase) loop with a bounded retry budget, so that a single mis-programmed cell cannot bring down the system: it is either re-programmed, erased and re-attempted, or declared a failure [1, 3, 4].

These three consequences are the design contract of the digital controller. The remainder of the report describes the RTL that implements that contract.

Chapter 3

System Architecture

In this chapter we want to presents the controller stack at the block-diagram level: the three RTL modules, the transaction-word format they agree on, and the host command set. Module-internal behavior - FSM state graphs, pulse-train arithmetic, the LUT-driven programming path - is deferred to Chapter 4.

3.1 Top-Level Organization

The stack sits between a host CPU and the analog Y-Flash core. The host presents a 32-bit data payload and a multi-bit command token; the controller decodes that traffic, maintains the closed-loop programming and verification policy, stages inference pixels, and drives the analog core over a structured output word and a pulse-mode bus that selects the active operation (read, program, erase, or inference). The top level module organization is shown in Figure 3.1.

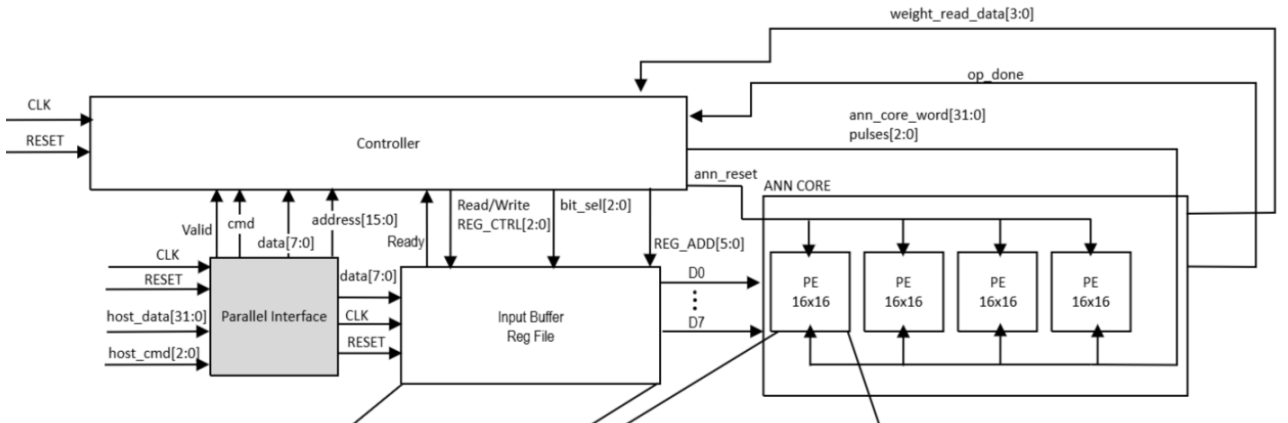


Figure 3.1: Top-level block diagram. The host drives a transaction word and a command token into the combinational *Parallel Interface*, which qualifies the packet and produces a validated bus consumed by the *ANN Controller*. The controller sequences commands toward the analog ANN core over a packed core word and a pulse-mode bus, and it simultaneously drives the *Input Buffer* that stores inference pixels and the expected-weight nibble used during verify. A single status bit from the core (*op_done*) closes the loop in the program and erase sub-FSMs.

Parallel Interface. A purely combinational host decoder. It slices the host payload into a data byte and a one-hot address tail, decodes the tail into a compact packed address, forwards the command token, and computes a single *valid* qualifier that asserts only when the command is not idle *and* every field of the one-hot tail is structurally legal. Invalid packets never propagate past the interface, so the controller can assume legal addressing whenever *valid* is high. Lifting all packet-level validation into a single combinational stage concentrates the protocol contract in one location and keeps the downstream FSM simpler.

ANN Controller. The only sequential module in the stack. The controller hosts a nine-state main FSM (idle, reset, program, verify, read, erase, collect, compute, result) plus three cooperating sub-FSMs that handle programming, verification, and erase. Two registered captures latch the host-supplied address and payload on the transition out of idle, so multi-cycle operations are immune to subsequent host activity. One large combinational block drives every module output; two clocked blocks register the FSM states, counters, and captured registers.

Input Buffer. A synchronous register-array store that plays two roles. During inference, it accumulates a pixel window written during *collect* cycles and exposes the pixels through eight parallel bit-serial lanes during *compute*. The intended inference workload is image classification on the 8×8 MNIST dataset, whose pixels are streamed eight per beat through the buffer; consequently, the window width and lane count are sized to accommodate the 8×8 input space. During programming, the same buffer holds the expected-weight 4-bit nibble for each cell so the verify comparator can fetch it without an extra host transaction. A single physical store serving both roles saves area and simplifies addressing inside the FSM.

3.2 Host-to-Core Transaction Word

The 32-bit transaction word combines a payload byte with four concatenated one-hot address fields, summarized in Table 3.1 and illustrated in Figure 3.2. The same layout is used at the host input and the core output; internally the controller operates on a compact 16-bit packed binary representation of the same address.

Table 3.1: Fields of the host-to-core transaction word.

Bits	Width	Field	Meaning
[31:24]	8	Payload	Quantized weight nibble (lower half) on programming, or one pixel byte on inference.
[23:20]	4	<i>PE</i> (one-hot)	Processing-element / block select.
[19:16]	4	<i>SA</i> (one-hot)	Sub-array / sub-block select.
[15:8]	8	<i>col_ID</i> (one-hot)	Column select within the sub-block.
[7:0]	8	<i>row_ID</i> (one-hot)	Row select within the sub-block.

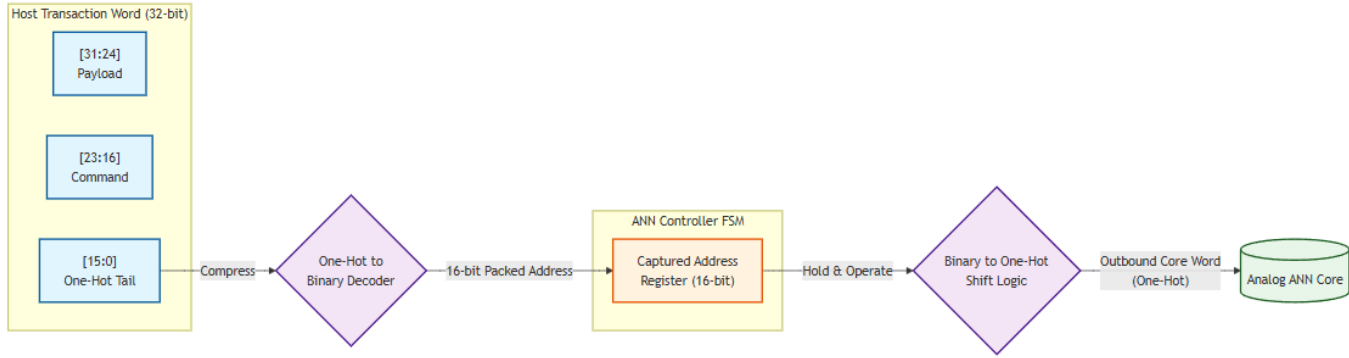


Figure 3.2: Pictorial layout of the 32-bit transaction word and the internal round-trip used by the controller: the host one-hot tail is decoded into a 16-bit packed binary address, the FSM captures and operates on the packed address, and the outbound core word is rebuilt by single-bit left-shifts into the one-hot domain just before reaching the core.

Why one-hot at the boundary. The one-hot encoding at the host interface mirrors how the analog core physically selects a cell: only the row and column whose deselect signal is released conduct, while every other device in the array is held in its undisturbed state (Section 2.2). Aligning the host-visible transaction format with the device-level selection mechanism allows the address to pass through the stack with minimal transformation while still being structurally validated at the boundary.

3.3 Command Set

The host command tokens are enumerated in Table 3.2. Each command class drives a specific main-FSM branch and a distinct pulse-mode encoding for the duration of the active phase.

Table 3.2: Host command set. Each command class is encoded as a 3-bit token on the host `cmd` input; the same 3-bit value is re-used at the core-facing pulse-mode bus, so command decode at the boundary and pulse-mode generation at the core share a single encoding space.

Command	Encoding	Meaning
HIZ	3'b000	Idle. Host bus is inactive; no transaction starts. The same encoding drives the pulse-mode bus to high impedance between bursts.
READ	3'b001	Read the addressed cell over a read pulse train.
PROG	3'b010	Program the addressed cell with the quantized weight in the payload; enters the closed-loop program–verify–erase policy.
ERASE	3'b011	Host-initiated erase of the addressed cell. Distinct from the verify-initiated erase taken on an over-programmed readback.
INF	3'b100	Inference. Pixel bytes are streamed over consecutive beats; the controller stages them in the buffer and then runs a compute phase.

3.4 Module Partitioning Philosophy

The three-module decomposition is a deliberate architectural partition, not a code-organization choice. Each module has a single responsibility with a narrow, structurally validated interface:

the interface is purely combinational and is the only module that touches host-side encoding; the controller is the only module that contains state, so every sequential decision in the stack is concentrated in one place; the input buffer is a synchronous data store with no knowledge of the high-level operation, exposing a small mode-qualified port set.

The benefit is twofold. First, each module can be verified in isolation against a focused contract - one-hot validity for the interface, FSM transition correctness for the controller, output-and-retention checks for the buffer - before integration. Second, replacing or extending any one module in a future revision (adding a serial host interface, pipelining the controller, swapping the register array for SRAM) does not require touching the other two. Bit-level conventions are centralized in per-module SystemVerilog packages that the testbenches also reuse, so the protocol is captured in exactly one place.

Chapter 4

RTL Module Design

Now we will describe each module as an architectural unit: what it must do, how its datapath and control logic are organized to do it, and the trade-offs behind each major decision. The emphasis is on the *hardware block flow* (e.g. `PROG` → `VERIFY` → `ERASE` → `RE-PROG`) that implements the closed-loop recovery policy. Cycle-accurate transition equations, complete state graphs, and port lists are documented in the project repository (see Supplementary Material).

4.1 Parallel Interface

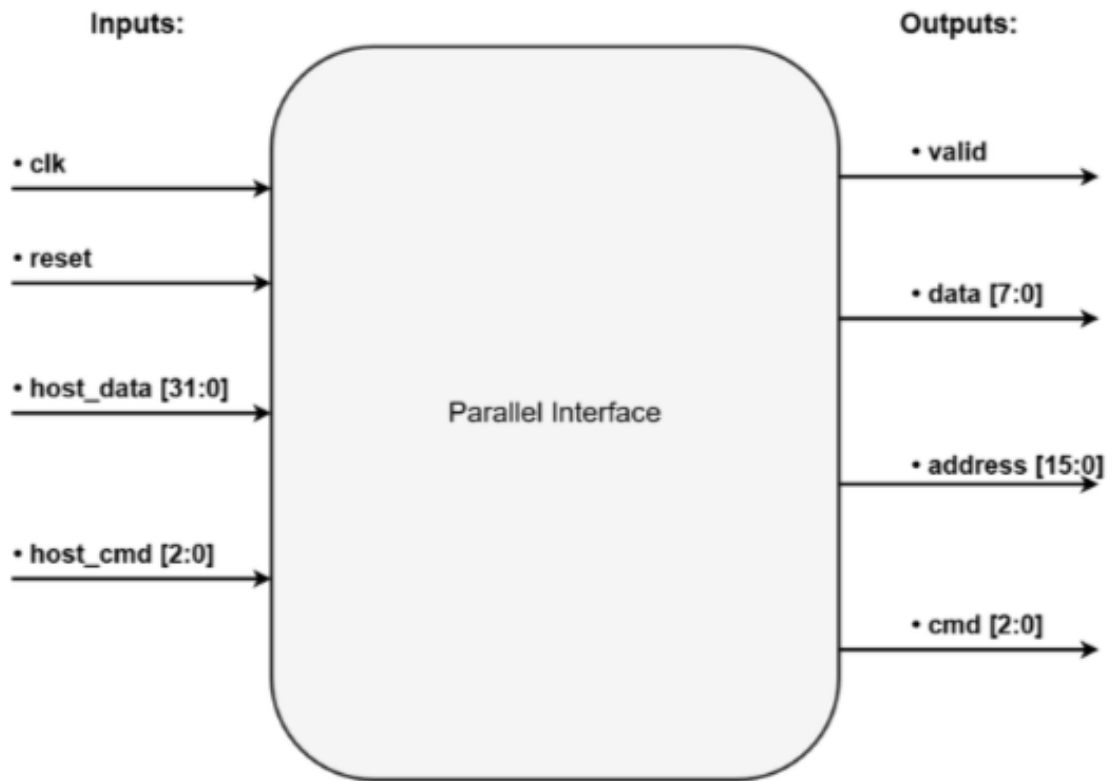


Figure 4.1: Parallel Interface I/O block diagram. The module exposes a host-side port set (`host_data`, `host_cmd`) and a controller-side port set (`address`, `data`, `cmd`, `valid`); all signals are combinational, so the entire block can be timed as a single decode stage.

The parallel interface is a purely combinational host decoder. It slices the host packet into a payload byte, a forwarded command token, a packed binary address, and a single qualification bit *valid*. *valid* asserts only when the host command is non-idle *and* each of the four one-hot fields (*PE*, *SA*, *col_ID*, *row_ID*) is *strictly* one-hot - all-zero and multi-hot patterns are both rejected. Figure 4.2 shows how the 32-bit host transaction word is sliced at the interface

boundary, and Figure 4.3 shows the packed binary address that the downstream controller actually sees.

Host Data: `host_data[31:0]`

Data	PE	SA	Column	Row
[31:24] 8 bit payload	[23:20] 4 bit one-hot	[19:16] 4 bit one-hot	[15:8] 8 bit one-hot	[7:0] 8 bit one-hot

Figure 4.2: 32-bit host transaction word split. The high byte carries the payload (weight nibble or pixel byte); the remaining 24 bits are partitioned into four concatenated one-hot fields - *PE* (4 bits), *SA* (4 bits), *col_ID* (8 bits), and *row_ID* (8 bits) - each of which must be *strictly* one-hot for the interface to assert `valid`.

Translated Address: `address[15:0]`

Unused	Block	Sub-block	Column	Row
[15:10] 6 bit = 0	[9:8] 2 bit index	[7:6] 2 bit index	[5:3] 3 bit index	[2:0] 3 bit index

Figure 4.3: Translated, packed address format passed to the controller. Each one-hot field of the host word is reduced to its binary index, and the four indices are concatenated into a 16-bit packed address: *PE* occupies bits [15:12], *SA* bits [11:8], *col_ID* bits [7:4], and *row_ID* bits [3:0]. Storage and arithmetic inside the controller use this packed form; the one-hot encoding is rebuilt by single-bit left-shifts only at the core-facing output.

Concentrating packet validation in a single combinational stage is a deliberate decision: it lifts the entire “what if the address is illegal” class of edge cases out of the downstream FSM, leaving the controller free to assume single-cell selection whenever it begins a transaction. The tail decoder reduces each one-hot field to its binary index in a single combinational pass, producing a compact 16-bit packed address that the controller stores and operates on internally. The one-hot form is rebuilt by a single-bit left-shift per field at the controller output, only at the very last step before the core.

Why translate to packed binary. The translation pays for itself three ways. *Storage*: a binary index needs 2–3 bits where the one-hot field needs 4–8, so the captured address register and every downstream counter shrinks accordingly. *Arithmetic*: verify-side comparators, buffer addressing, and internal counters treat the cell index as an integer, which is a few bits of slicing on the packed form versus non-trivial encoders interspersed through the FSM on the one-hot form. *Validation as a one-time event*: once the boundary has accepted the packet, the controller can assume structural legality and does not re-check.

4.2 ANN Controller

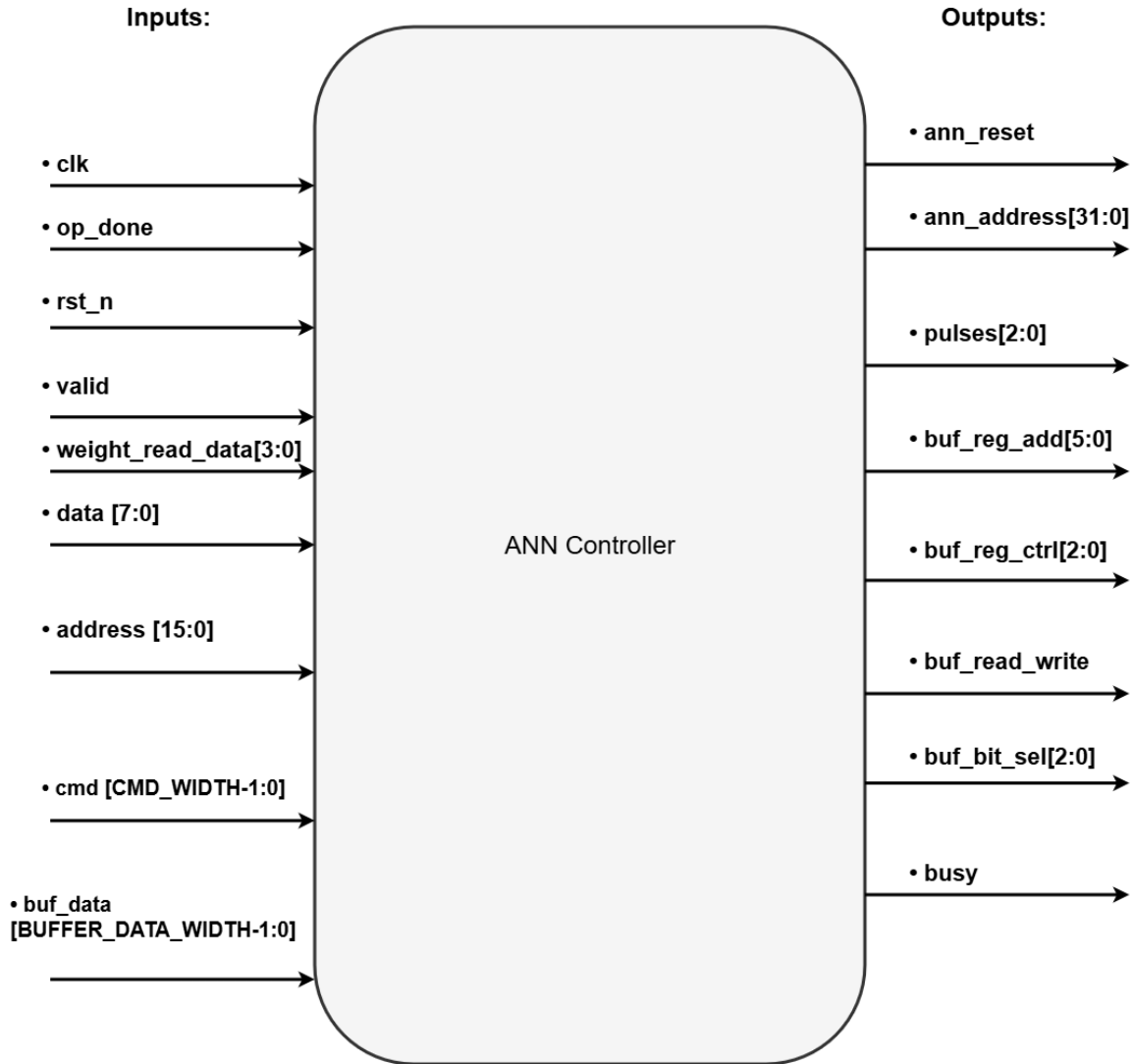


Figure 4.4: ANN Controller I/O block diagram. Host-side inputs (`address`, `data`, `cmd`, `valid`) arrive pre-validated from the parallel interface; analog-side inputs (`op_done`, `weight_read_data`) close the program/erase and verify loops. The controller emits the packed core word (`addr_out`, `data_out`), the three-bit pulse-mode bus (`pulses`), the buffer-control port set, and a small bank of status flags (`erase_retry_exhausted`, `verify_failed`). Note that this diagram shows a high-level abstraction; not all signals or ports are explicitly represented. Comprehensive signal and port tables are provided in the project repository.

As mentioned earlier (Section 3.1), the controller is the only sequential block in the stack: the parallel interface decodes combinatorially and the input buffer is a passive register-array store, so every multi-cycle decision in the system - command sequencing, closed-loop recovery, pulse-train shaping - is concentrated in this module. It is built around a central combinational block that drives all module outputs and produces every next-state value, plus two clocked blocks that register the main FSM state, the sub-FSM states, the pulse counters, and the captured host bus. The “one big combinational block, two clocked blocks” style gives the synthesis tool a single coherent picture of next-state and output logic and keeps the timing of

every output predictable from the same source.

4.2.1 Registered Capture of Host Data

The host-supplied address and payload byte are latched on the transition out of idle. Outside that single transition, both registers retain their values, so every multi-cycle state (program, verify, read, erase) operates on a stable snapshot of the transaction and is immune to subsequent host activity. The inference path is the only deliberate exception: during **COLLECT**, the live host data byte is forwarded so that the host can stream a sequence of pixel bytes across consecutive beats.

4.2.2 Address Round-Trip and Payload Multiplexing

When the controller is active it decomposes the captured packed address back into the four binary indices and rebuilds the outbound transaction word combinationally: a single-bit left-shift by each index produces the four one-hot fields the analog core expects. In idle the outbound word is forced to zero so that the core sees a quiet bus between operations. The payload byte multiplexer selects the registered byte during program/verify/read/erase (so the same value is presented for the entire multi-cycle operation), the live host byte during the collect phase of inference, and zero in every other state.

4.2.3 Main FSM: Hardware Block Flow

The main FSM has nine states. The combinational block applies a uniform set of default outputs and then overrides them per-state in a **unique case** body, so that every state has a single, well-defined set of bus values. The architectural flow is presented in two complementary views: Figure 4.5 isolates the command-decode decision taken in **S_IDLE** (which selects the operation branch), and Figure 4.6 shows the full set of nine states together with the closed-loop recovery policy that links them. Cycle-by-cycle transition rules are catalogued in the repository.

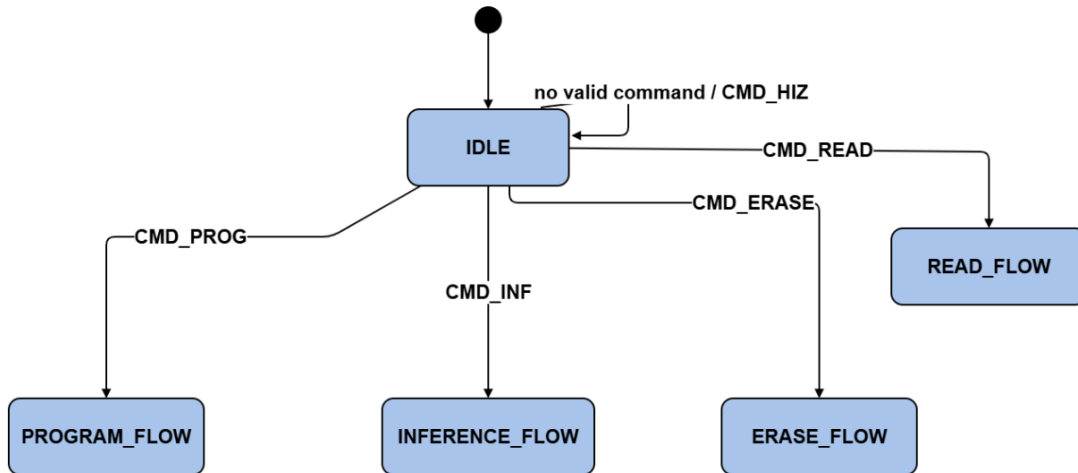


Figure 4.5: Command decode and state routing from **S_IDLE**. While **valid** is low or **cmd** equals **CMD_HIZ**, the FSM holds in **S_IDLE**; otherwise a **unique case** on **cmd** routes into exactly one operation branch (**S_PROGRAM**, **S_ERASE**, **S_READ**, or **S_COLLECT_DATA**). This high-level view highlights the routing logic; the full nine-state transition set is detailed in the subsequent architecture sections.

Table 4.1: Top-level command decode from `S_IDLE`. Each branch is entered combinationaly on the first cycle that `valid` is high; subsequent transitions of the destination state are documented in Table 4.2.

State	Purpose	Transition Condition & Next State
<code>S_IDLE</code>	Quiescent; combinationally decodes <code>cmd</code> when <code>valid</code> is asserted, latches the host address and payload, and routes into the matching operation branch.	<code>!valid</code> or <code>CMD_HIZ</code> $\rightarrow$ <code>S_IDLE</code> ; <code>CMD_PROG</code> $\rightarrow$ <code>S_PROGRAM</code> ; <code>CMD_ERASE</code> $\rightarrow$ <code>S_ERASE</code> ; <code>CMD_READ</code> $\rightarrow$ <code>S_READ</code> ; <code>CMD_INF</code> $\rightarrow$ <code>S_COLLECT_DATA</code> .
<code>S_PROGRAM</code> (entry)	Closed-loop programming branch; runs the program sub-FSM and hands off to verify on completion.	See Table 4.2.
<code>S_ERASE</code> (entry)	Erase branch; runs the erase sub-FSM. Host/verify flag distinguishes host-initiated from verify-initiated entry.	See Table 4.2.
<code>S_READ</code> (entry)	Single-cell read branch; drives the read pulse train.	See Table 4.2.
<code>S_COLLECT_DATA</code> (entry)	Inference branch; streams pixel bytes into the buffer and advances to compute and result.	See Table 4.2.

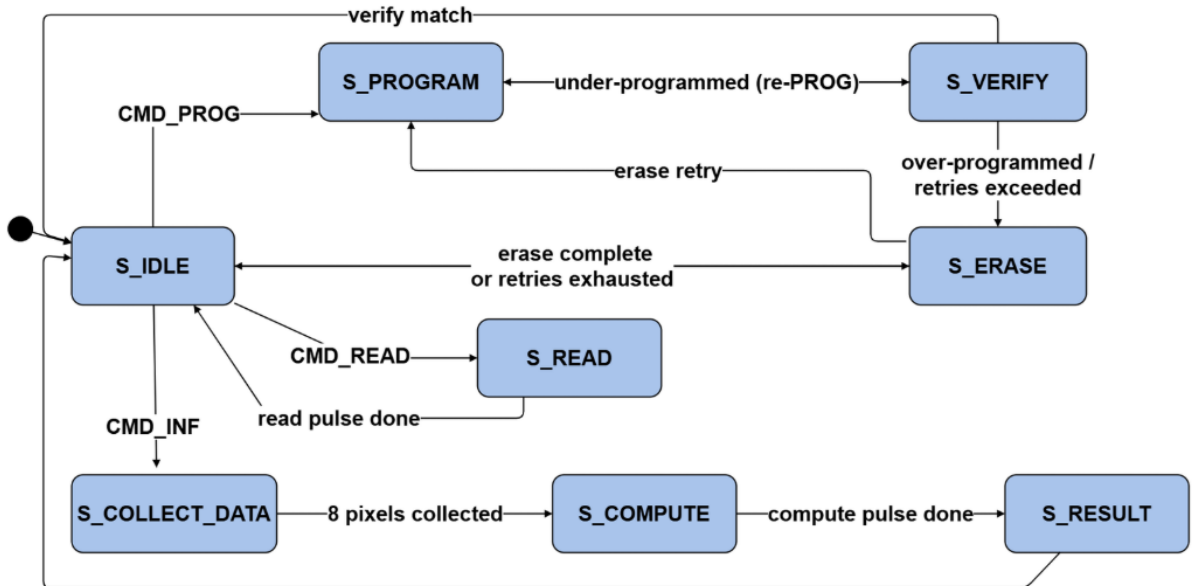


Figure 4.6: The 9 states and the closed-loop recovery policy. Edges out of `S_VERIFY` and `S_ERASE` summarize the decisions taken in the sub-FSMs - match, under-programmed, over-programmed, retry-exhausted - which implement the `PROG` $\rightarrow$ `VERIFY` $\rightarrow$ `ERASE` $\rightarrow$ `RE-PROG` recovery policy.

Table 4.2: Main-FSM states. The `PROG` $\rightarrow$ `VERIFY` $\rightarrow$ `ERASE` $\rightarrow$ `RE-PROG` recovery policy is implemented by the branches out of `S_VERIFY` and `S_ERASE`; sub-FSM-internal sequencing is documented in Tables 4.3 to 4.5.

State	Purpose	Transition Condition & Next State
<code>S_IDLE</code>	Quiescent; decodes host command and latches captured address/payload on the transition out of idle.	<code>!valid</code> or <code>CMD_HIZ</code> $\rightarrow$ <code>S_IDLE</code> ; otherwise routes per Table 4.1.
<code>S_RESET</code>	One-cycle pulse to the core-reset output; currently unreachable from idle and reserved as the attachment point for a future global-reset command (Chapter 7).	Unconditional $\rightarrow$ <code>S_IDLE</code> .
<code>S_PROGRAM</code>	Drives the address onto the core word and runs the program sub-FSM through its physical phases (<code>HIZ</code> , <code>SELECT</code> , <code>WRITE</code> , <code>WAIT_ACK</code> , <code>COMPLETE</code>).	On <code>PROG_COMPLETE</code> $\rightarrow$ <code>S_VERIFY</code> (the closed-loop entry to verify).
<code>S_VERIFY</code>	Holds the buffer in weight-read mode and runs the verify sub-FSM; the <code>CHECK</code> state compares readback against the expected weight.	<code>Match</code> $\rightarrow$ <code>S_IDLE</code> ; under-programmed and retries available $\rightarrow$ <code>S_PROGRAM</code> (nudge retry); under-programmed and retries exhausted $\rightarrow$ <code>S_ERASE</code> (verify flag set); over-programmed $\rightarrow$ <code>S_ERASE</code> (verify flag set).
<code>S_ERASE</code>	Runs the erase sub-FSM and drives the erase encoding on the pulse-mode bus. Host/verify flag distinguishes the two entry contexts.	On <code>ERASE_COMPLETE</code> , branches per Table 4.6: host-initiated $\rightarrow$ <code>S_IDLE</code> ; verify-initiated with retries available $\rightarrow$ <code>S_PROGRAM</code> ; verify-initiated with retries exhausted $\rightarrow$ <code>S_IDLE</code> .
<code>S_READ</code>	Presents the captured address and the read encoding for the duration of the read window.	<code>pulse_done</code> $\rightarrow$ <code>S_IDLE</code> .
<code>S_COLLECT_DATA</code>	Streams up to eight live pixel bytes into the buffer at successive slots derived from the pixel counter.	After eight beats (pixel-count threshold) $\rightarrow$ <code>S_COMPUTE</code> .
<code>S_COMPUTE</code>	Switches the buffer to bit-serial output mode and advances <code>bit_sel</code> from zero to seven so the analog core sees one bit of each of the eight pixels in parallel per cycle.	<code>pulse_done</code> $\rightarrow$ <code>S_RESULT</code> .
<code>S_RESULT</code>	One-cycle wrap-up; switches the buffer to result-out mode.	Unconditional $\rightarrow$ <code>S_IDLE</code> .

Any out-of-range state value falls through the `unique case default` arm and returns the FSM to `S_IDLE`, providing a fail-safe path against a glitched state register.

4.2.4 Sub-FSMs and the Closed-Loop Recovery Policy

Three sub-FSMs run inside the main FSM and together implement the closed-loop recovery policy. Their states correspond to the physically distinct phases of a Y-Flash operation, so each sub-state has a natural hardware interpretation rather than an abstract control role.

4.2.4.1 Program Sub-FSM

A five-state sequence that mirrors the physical phases of a programming operation: **HIZ** (bus settle, one cycle high-impedance) → **SELECT** (drive address, wait for analog *op_done* acknowledging cell selection, simultaneously write the expected weight nibble into the buffer for the subsequent verify) → **WRITE** (drive the program encoding, advance the pulse counter through the configured pulse train, exit on *pulse_done*) → **WAIT_ACK** (idle bus while the in-cell transient settling, exit on second *op_done*) → **COMPLETE** (hand control to the main FSM with next-state = verify, reset the sub-FSM to **HIZ**). The pulse train of the **WRITE** state is selected by the LUT-based shaper on the first attempt and falls back to a short single-cycle nudge train on retries (Section 4.2.5).

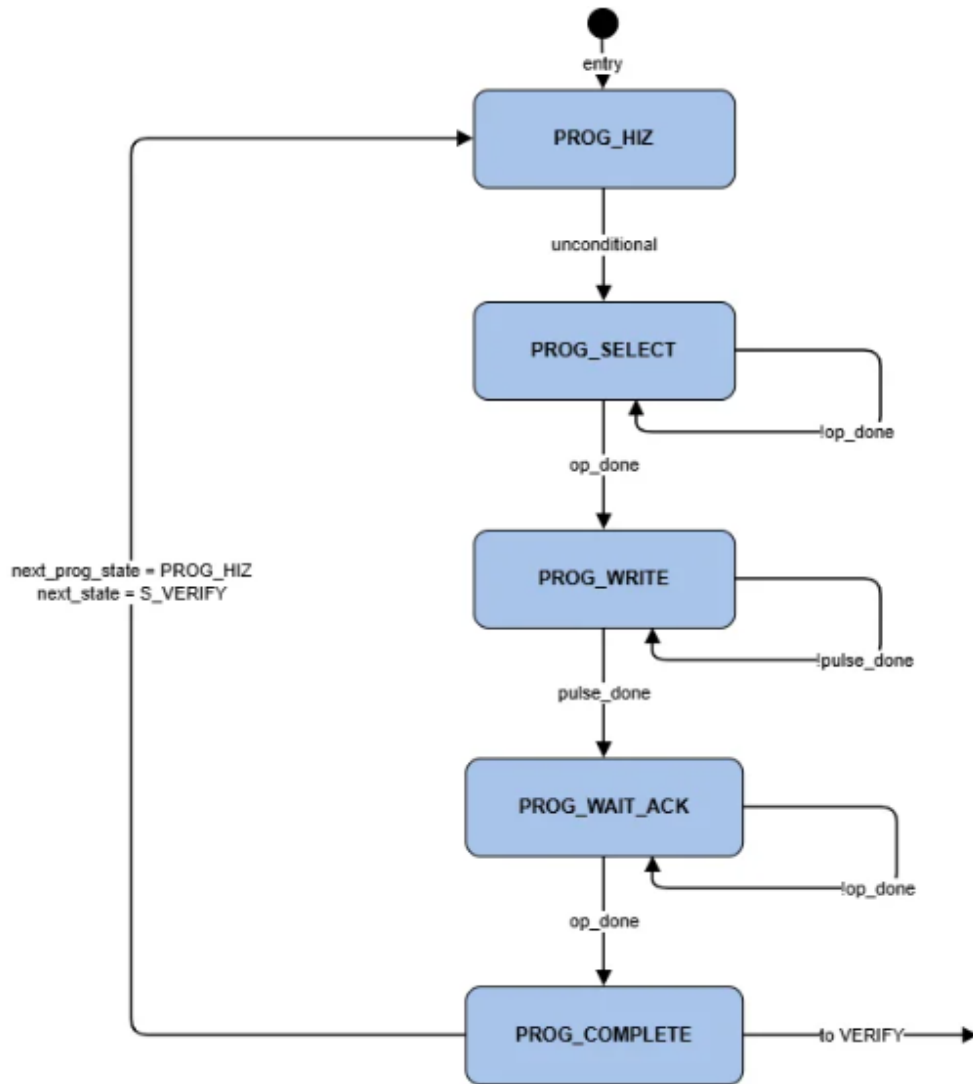


Figure 4.7: Program sub-FSM. Five sub-states map onto the physical phases of a Y-Flash programming operation: **HIZ**, **SELECT**, **WRITE**, **WAIT_ACK**, **COMPLETE**.

Table 4.3: Program sub-FSM states. The pulse train of `PROG_WRITE` is selected by the LUT-based shaper on the first attempt and falls back to a short single-cycle nudge train on retries (Section 4.2.5).

State	Purpose	Transition Condition & Next State
<code>PROG_HIZ</code>	One-cycle bus settle; pulse-mode bus held at high impedance.	Unconditional $\rightarrow$ <code>PROG_SELECT</code> .
<code>PROG_SELECT</code>	Drive the address onto the core word, wait for the analog <code>op_done</code> acknowledging cell selection, and simultaneously write the expected 4-bit weight nibble into the buffer for the subsequent verify.	<code>op_done</code> $\rightarrow$ <code>PROG_WRITE</code> .
<code>PROG_WRITE</code>	Drive the program pulse-mode encoding while the pulse counter advances through the configured pulse train (LUT macro-repeat on the first attempt, single-cycle nudge on retries).	<code>pulse_done</code> $\rightarrow$ <code>PROG_WAIT_ACK</code> .
<code>PROG_WAIT_ACK</code>	Idle bus while the in-cell programming transient settles on the analog side.	Second <code>op_done</code> $\rightarrow$ <code>PROG_COMPLETE</code> .
<code>PROG_COMPLETE</code>	Hand control back to the main FSM with next main state = <code>S_VERIFY</code> and reset the program sub-FSM.	Unconditional $\rightarrow$ <code>PROG_HIZ</code> (and main FSM $\rightarrow$ <code>S_VERIFY</code>).

4.2.4.2 Verify Sub-FSM and the Recovery Branch

A four-state sequence: `IDLE` (entry, with a short-circuit path to `CHECK` for the degenerate single-cycle read window) $\rightarrow$ `WAIT` (pulse-train active, increment index counter each cycle until the configured length is reached) $\rightarrow$ `CHECK` (apply the recovery decision, below) $\rightarrow$ `DONE` (on success, return the main FSM to idle).

`CHECK` is the architectural heart of the controller: it compares the analog-side readback against the expected weight in the input buffer and branches into one of three outcomes - match, under-programmed, over-programmed - that drive the entire recovery policy. The branch into `ERASE` on an over-programmed readback follows directly from the device physics: there is no monotonic way to subtract injected charge with a program pulse on a Y-Flash cell, so the only valid recovery from over-programming is to erase the cell and reprogram it from a known low-conductance state. The detailed branch logic is documented row-by-row in Table 4.4.

The per-cell retry counter is cleared on a successful verify, on an over-programmed readback (the failure mode is not retry exhaustion), and on completion of a verify-initiated erase. Together these resets scope the counter to a single cell at a time.

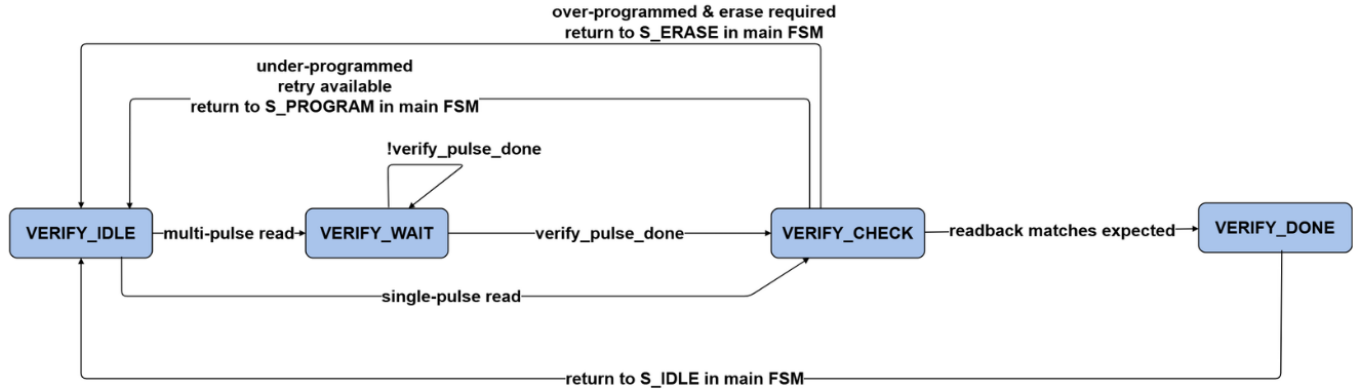


Figure 4.8: CHECK state branching logic: match, under-programmed, over-programmed. The three outcomes drive the closed-loop recovery policy - success returns to idle, under-programmed retries (or falls back to erase when the per-cell retry budget is exhausted), and over-programmed triggers an erase followed by reprogramming.

Table 4.4: Verify sub-FSM states. The four states correspond to the readback pulse-train window (IDLE, WAIT), the comparison decision (CHECK), and the closing handshake to the main FSM (DONE). Branches in CHECK reach back into S_PROGRAM or S_ERASE of the main FSM (Table 4.2).

State	Purpose	Transition Condition & Next State
VERIFY_IDLE	Entry state for the verify pulse-train window; provides a short-circuit path into CHECK for the degenerate single-cycle read configuration.	Pulse-train length $\leq 1 \rightarrow$ VERIFY_CHECK; otherwise $\rightarrow$ VERIFY_WAIT.
VERIFY_WAIT	Pulse-train active; the verify index counter is incremented each cycle until the configured pulse-train length is reached.	Counter reaches configured length $\rightarrow$ VERIFY_CHECK.
VERIFY_CHECK	Compare the analog readback against the expected 4-bit weight held in the buffer and branch into one of three outcomes that drive the recovery policy.	Match (readback = expected) $\rightarrow$ VERIFY_DONE; under-programmed (readback < expected) with retries available $\rightarrow$ main FSM S_PROGRAM (nudge retry); under-programmed and retries exhausted $\rightarrow$ main FSM S_ERASE (verify flag set); over-programmed (readback > expected) $\rightarrow$ main FSM S_ERASE (verify flag set).
VERIFY_DONE	Successful verify; return the main FSM to idle and clear the per-cell retry counter.	Unconditional $\rightarrow$ VERIFY_IDLE (and main FSM $\rightarrow$ S_IDLE).

4.2.4.3 Erase Sub-FSM

A five-state sequence whose structure mirrors the program sub-FSM, reflecting the symmetric structure of the two physical operations on the device: HIZ $\rightarrow$ SELECT $\rightarrow$ PULSE (erase encoding on the pulse-mode bus) $\rightarrow$ WAIT_ACK (in-cell discharge transient) $\rightarrow$ COMPLETE. The exit from COMPLETE is governed by the host/verify flag and the global retry counter; the per-state breakdown is given in Table 4.5, and the exit-decision matrix is summarized in Table 4.6.

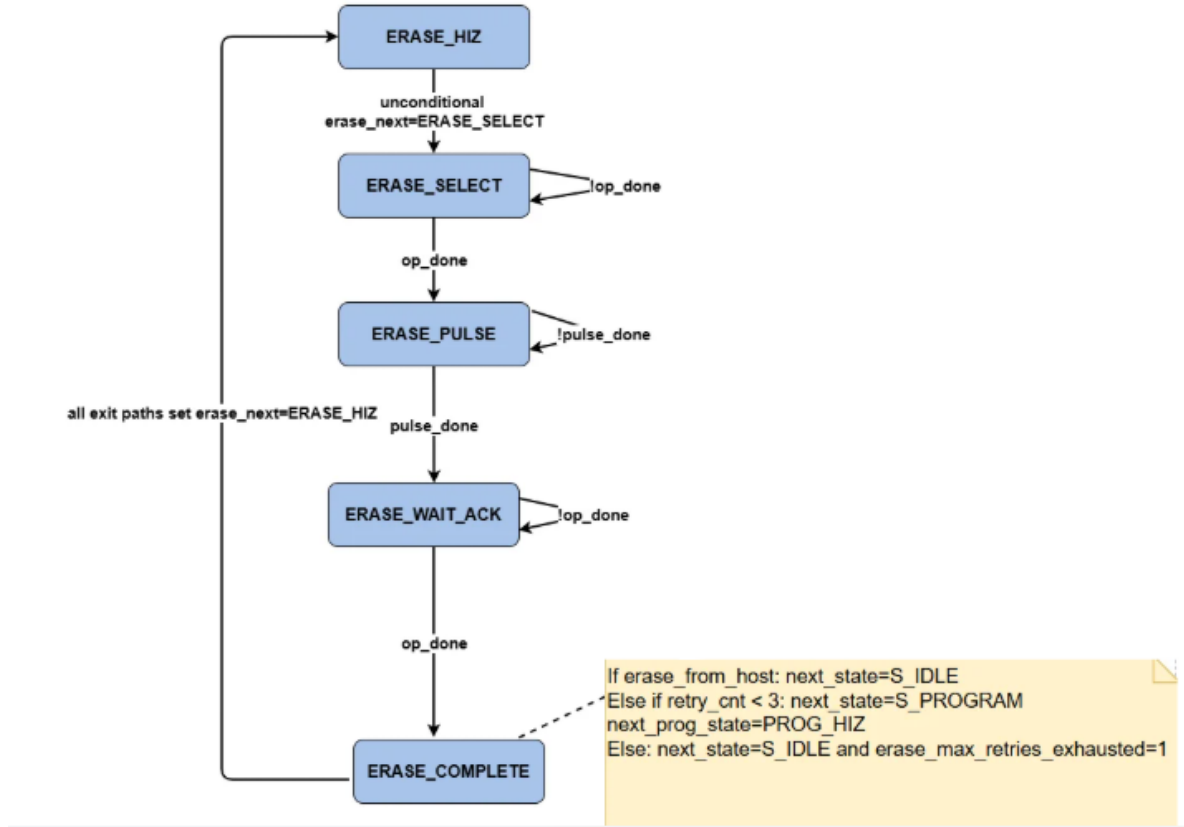


Figure 4.9: Erase sub-FSM. The state graph is symmetric with the program sub-FSM (Figure 4.7), reflecting the equivalent physical sequencing of the two device operations. The host/verify flag set on entry steers the exit from ERASE_COMPLETE between a return to idle (host-initiated, or verify-initiated with the retry budget exhausted) and a re-entry into S_PROGRAM (verify-initiated with retries available).

Table 4.5: Erase sub-FSM states. The structure mirrors Table 4.3 state-for-state; the only substantive difference is the closing branch in `ERASE_COMPLETE`, detailed in Table 4.6.

State	Purpose	Transition Condition & Next State
<code>ERASE_HIZ</code>	One-cycle bus settle; pulse-mode bus held at high impedance.	Unconditional $\rightarrow$ <code>ERASE_SELECT</code> .
<code>ERASE_SELECT</code>	Drive the address onto the core word and wait for the analog <code>op_done</code> acknowledging cell selection.	<code>op_done</code> $\rightarrow$ <code>ERASE_PULSE</code> .
<code>ERASE_PULSE</code>	Drive the erase pulse-mode encoding while the pulse counter advances through the configured erase pulse train.	<code>pulse_done</code> $\rightarrow$ <code>ERASE_WAIT_ACK</code> .
<code>ERASE_WAIT_ACK</code>	Idle bus while the in-cell band-to-band tunneling discharge transient settles.	Second <code>op_done</code> $\rightarrow$ <code>ERASE_COMPLETE</code> .
<code>ERASE_COMPLETE</code>	Closing branch; selects the next main state from the host/verify flag and the global retry budget.	Host-initiated $\rightarrow$ <code>S_IDLE</code> ; verify-initiated and retries available $\rightarrow$ <code>S_PROGRAM</code> ; verify-initiated and retries exhausted $\rightarrow$ <code>S_IDLE</code> . See Table 4.6.

Table 4.6: Exit rules from `ERASE_COMPLETE`.

Condition	Next main state	Effect
Host-initiated erase	<code>S_IDLE</code>	One-shot host operation; no retry.
Verify-initiated, retries available	<code>S_PROGRAM</code>	Recovery loop; re-enters the program state. Global retry counter increments.
Verify-initiated, retries exhausted	<code>S_IDLE</code>	Cell declared a hard failure; the <code>erase_retry_exhausted</code> flag is asserted to the upper layer.

4.2.5 Pulse Generation and LUT-Based Programming

Every active state that touches the analog core is pulse-shaped. The controller maintains three coherent artifacts: a combinational per-state pulse-length value, a registered cycle counter, and a combinational pulse-completion signal. Outside the active states the pulse-length is zero and the completion signal is held low, so no spurious completion can ever advance the FSM.

The pulse-mode output to the core defaults to its idle (high-impedance) encoding. For each active state, a per-state Boolean - derived either from a standard burst pattern or, for the LUT-based programming case, from a macro-repeat pattern - gates the active encoding onto the bus during burst cycles and falls back to idle during inter-burst windows. The bus is therefore a time-multiplexed burst train and never enters an undefined state. The physical structure of a single burst is illustrated in Figure 4.10.

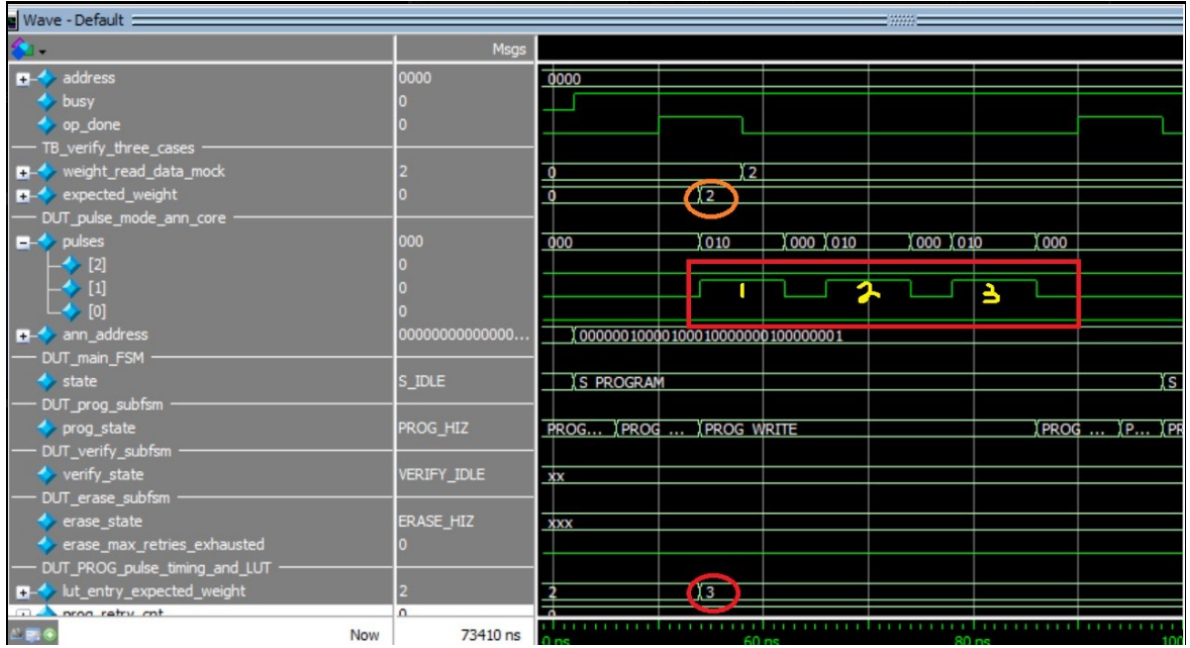


Figure 4.10: Physical pulse-train structure on the three-bit pulse-mode bus. A burst is the concatenation of `pulse_num` discrete active pulses, each T_{prog} cycles wide, separated by single-cycle HIZ (3'b000) idle gaps. The active mode (3'b010 for program in the illustration) is driven only while the gating Boolean is high; between pulses, and after the burst completes, the bus reverts to HIZ. Total burst length is therefore $pulse_num \cdot T_{prog} + (pulse_num - 1) \cdot PULSE_GAP$. *Numeric example, LUT-driven first PROG:* for an expected weight of $w_e = 2$ the LUT emits a macro-repeat count $R = 3$ (circled in red). With the baseline configuration $T_{prog} = 2$, $pulse_num = 1$, and $PULSE_GAP = 1$, the inner macro is two cycles long; the controller plays $R = 3$ back-to-back copies of that macro separated by single-cycle HIZ gaps (annotated 1, 2, and 3), yielding a total `PROG_WRITE` duration of $3 \cdot 2 + 2 \cdot 1 = 8$ cycles, of which 6 are active (`PULSE_MODE_PROG` on the bus) and 2 are idle HIZ gaps between copies.

Data-driven first programming attempt. On the first programming attempt for a given cell, the controller shapes the program pulse based on the expected weight value. A look-up table indexed by the four-bit expected weight returns a macro-repeat count: how many copies of the standard program burst to concatenate, separated by idle windows. The LUT is loaded from a memory image at elaboration time and can be replaced without modifying the RTL - this is the mechanism by which the controller absorbs device-characterization data without re-synthesis. Figure 4.11 shows the resulting structure.

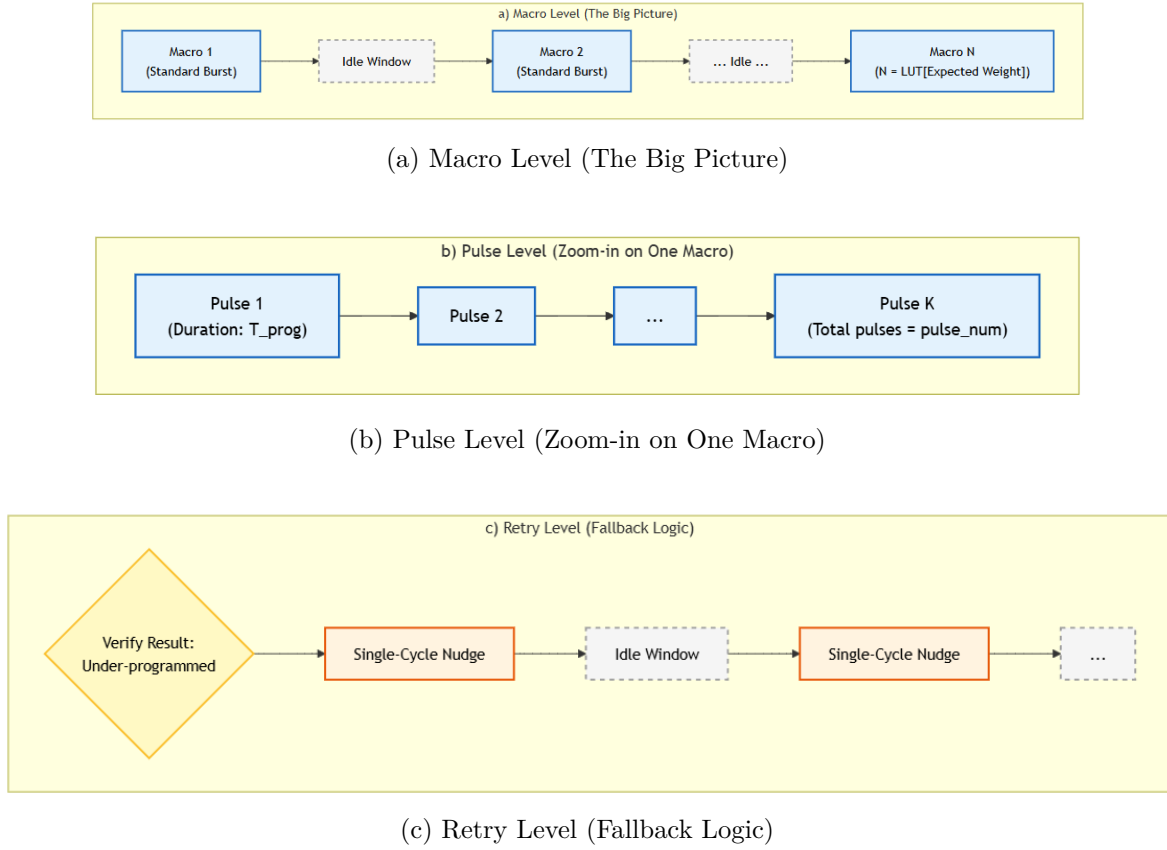


Figure 4.11: LUT-based macro-repeat structure for the programming attempts. **(a)** For the first attempt, the controller concatenates a number of macros given by the LUT entry for the expected weight, separated by idle windows. **(b)** The inner macro detail, showing one standard burst of discrete pulses. **(c)** Retry attempts on the same cell switch to a single-cycle nudge train, on the assumption that the first attempt has already moved the cell most of the way to the target.

The architectural justification for the LUT path is three-fold: per-weight pulse shaping can be re-calibrated by editing the LUT image without revisiting the RTL, which gives the device-characterization team a clean handoff point; the combination of a LUT-scaled first attempt with a short retry pulse is consistent with the device-physics observation that incremental programming pulses are more controllable than repeated large ones [3]; and if the LUT mechanism is disabled at elaboration, the controller falls back to a fixed-length program burst and the recovery policy still operates - the first-attempt success rate simply drops.

4.2.6 Inference Path

The inference path is a four-state sequence: *collect-data* (eight pixel bytes streamed into the buffer at row-major addresses derived from the running pixel count), *compute* (buffer switched to bit-serial mode, bit-select line advanced from zero to seven so the analog core sees one bit of each of the eight pixels in parallel per cycle), *result* (one-cycle wrap-up in result-out mode), and *idle*. Unlike the program/erase paths, inference does not consume *op_done*: it is a read-only operation with no in-cell transient that needs an explicit acknowledgement, so the duration of the compute phase is set entirely by the configured inference pulse length.

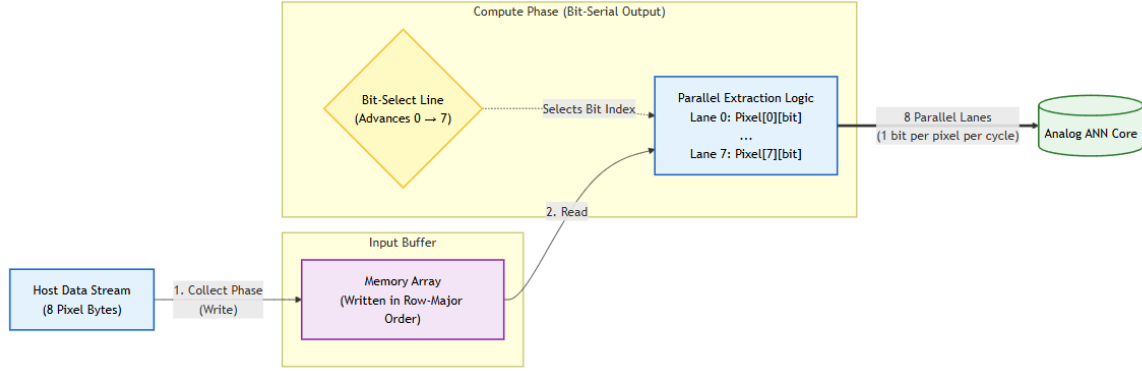


Figure 4.12: Inference dataflow. Eight pixel bytes are streamed in collect, written into the buffer in row-major order, then exposed bit-serially across eight parallel lanes during compute. The bit-select line advances from zero to seven so the analog core receives one bit of each pixel per cycle.

4.2.7 Recovery Loop, End to End

Figure 4.13 consolidates the closed-loop policy in a single picture: programming enters verify; verify either returns to idle on a match, re-enters programming on an under-programmed readback within the retry budget, or enters erase on either an over-programmed readback or retry-budget overflow; erase either returns to idle (host-initiated, or retry-budget overflow) or re-enters programming after a fresh erase. The diagram is the architectural answer to the write–verify problem of Section 2.3: every failure mode of an analog programming attempt is mapped to a well-defined digital recovery path, so that the controller delivers a stable weight matrix in the face of bounded device non-idealities.

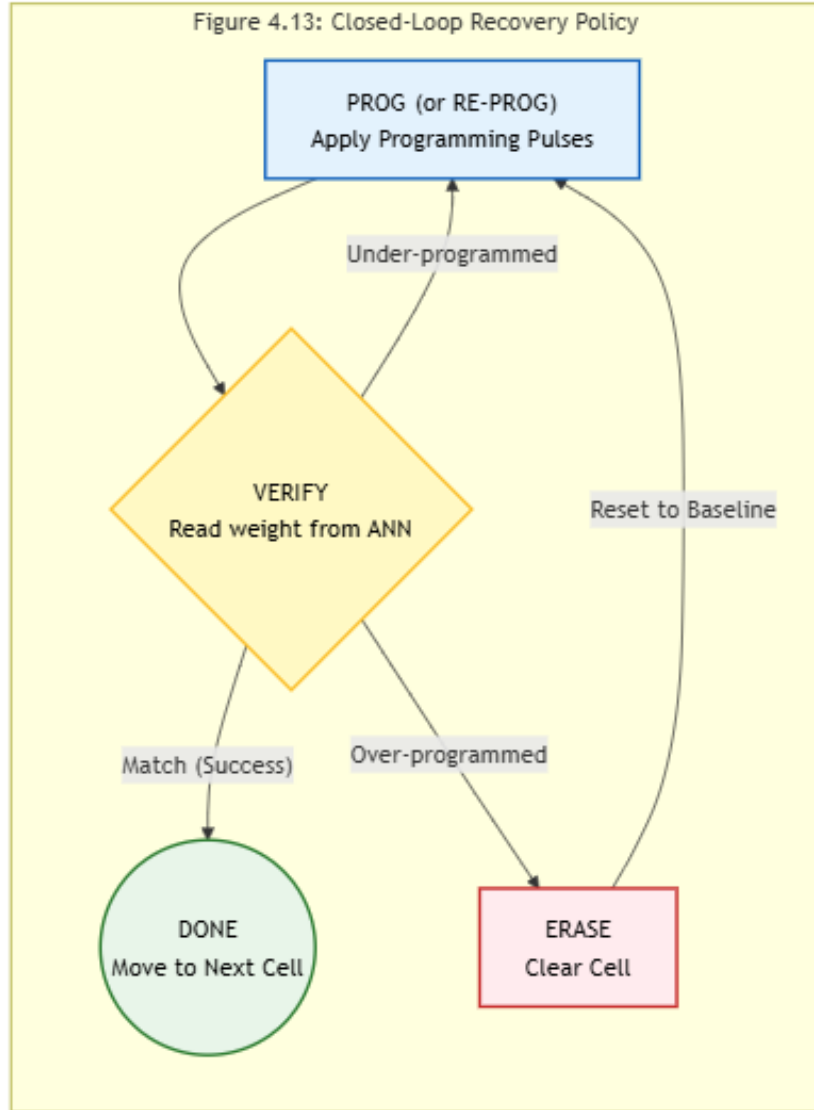


Figure 4.13: Closed-loop recovery policy. The PROG → VERIFY → ERASE → RE-PROG loop maps every analog failure mode to a deterministic digital recovery path.

This recovery policy is the central reason that the controller is more complex than a conventional write-path controller, and it shapes the verification environment of Chapter 5: every branch must be exercised in simulation, and the only practical way to do that without a physical analog core is to inject faults at the readback port.

Chapter 5

Verification Methodology

To establish rigorous confidence in the design, we have to answer two fundamental questions regarding our testing strategy: *what was the verification trying to prove*, and *how was it possible to prove it without access to the physical analog ANN core*? The answers justify a third claim, made forward: that the present functional simulation delivers enough confidence to hand the RTL to the analog phase of the project. Mechanical testbench details - file structure, helper classes, console-log formatting - are documented in the repository.

5.1 Verification Objectives

The controller has no datapath arithmetic of its own. What it *does* is translate host transactions into a stream of analog operations on a Y-Flash array and react correctly to the analog-side observables that come back. Verification therefore aims to establish three classes of claim:

1. **Static transaction-word contract.** For every legal host transaction, the controller produces a well-formed core word and the correct pulse-mode encoding: address fields bit-consistent with the host-supplied one-hot tail, payload forwarded faithfully, and the pulse-mode bus carrying the encoding appropriate to the active operation.
2. **Dynamic protocol compliance.** The FSM transitions through its states in the prescribed order, with the prescribed handshakes, without stalling. Programming advances `SELECT` $\rightarrow$ `WRITE` $\rightarrow$ `WAIT_ACK` $\rightarrow$ `COMPLETE`; verify follows programming; erase is entered from the correct path (host or verify) and exits to the correct main state; inference collects the right number of pixels before compute. Every protocol error here would manifest only over multiple cycles in interaction across sub-FSMs.
3. **Closed-loop recovery correctness.** The central architectural claim. Match $\rightarrow$ exit. Under $\rightarrow$ retry within budget, fail to erase otherwise. Over $\rightarrow$ erase, then retry within budget or terminate. Every branch of the recovery diagram in Section 4.2.7 is a verification obligation.

5.2 The Behavioral Mock: Standing in for the Analog ANN

The analog Y-Flash array is not part of this project, so verification substitutes it with a behavioral model that exposes the same digital boundary signals - the inbound core word and pulse-mode bus, the outbound readback line and *op_done* handshake. The mock captures only the digital contract at the boundary, not the analog physics; that is the right choice because the controller's correctness depends on those four signals and on nothing else. The integration is shown in Figure 5.1.

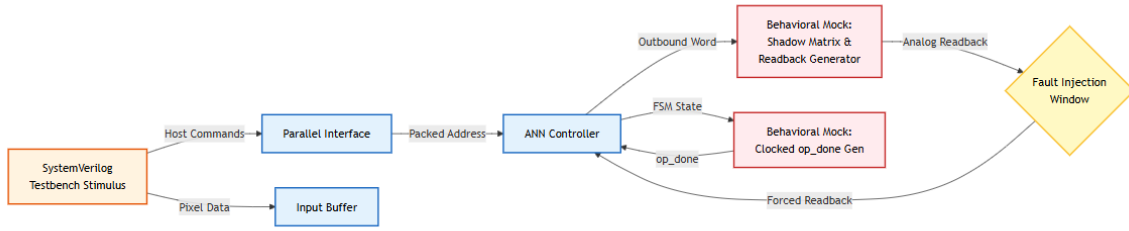


Figure 5.1: Verification environment. The three RTL modules are instantiated in their real host-to-core order; the analog ANN array is replaced by a behavioral mock with a shadow weight matrix, a combinational readback generator driven by the outbound core word, and a clocked *op_done* generator that follows the controller’s own sub-FSM. Fault injection at the readback line during the verify window forces the under- and over-programmed branches of the recovery FSM.

The mock has three pieces. The **shadow weight matrix** is a four-dimensional array indexed by the same (*PE*, *SA*, row, column) coordinates as the address tail. On every cycle in which the controller is in the active programming phase, the mock decodes the outbound core word and writes the quantized weight nibble carried in the payload byte into the corresponding shadow cell. A subsequent read returns that cell’s value. The **behavioral readback generator** continuously decodes the current outbound core word and selects the shadow cell whose value should appear on the readback line; with this default behavior alone, a well-behaved program always satisfies the verify comparator on the first attempt, so the recovery branches are never visited - which motivates the fault injection of Section 5.3. The ***op_done* generator** is a small clocked process that asserts the handshake whenever the controller is in a sub-state structurally waiting for it (*PROG_SELECT*, *PROG_WAIT_ACK*, *ERASE_SELECT*, *ERASE_WAIT_ACK*), with a small delay counter on pulse-driven phases to prevent the bench from stalling. This is testbench-only timing: it is not modeling a specific analog latency, only enough to let the FSM advance.

Why the mock is sufficient for confidence. Three observations justify the choice. *The controller only sees the digital contract*: every other physical attribute of the array - voltage levels, leakage, parasitic settling - is invisible to the FSM, so any controller bug that depends only on digital observables will be reproduced in simulation. *The mock matches the addressing convention exactly*: the shadow matrix is indexed by the same decomposition the RTL itself uses, eliminating the largest single source of mock-RTL drift. *The mock is composable*: swapping it for a higher-fidelity model (a Verilog-A cell model or an analog co-simulation harness) requires no change to either the controller or the testbench skeleton. The directed regressions written here will run against any model that conforms to the same digital boundary.

5.3 Forcing the Recovery Branches: Fault Injection

The most subtle architectural element of the verification environment is the fault-injection mechanism. Its purpose is to address the verification gap that the default mock leaves open: a well-behaved mock always returns the freshly programmed value, so the under- and over-programmed branches of the verify sub-FSM would never be visited and the entire closed-loop policy - the central architectural claim of the project - would remain unverified.

The mechanism overrides the readback value during a carefully gated window inside the verify phase. The override fires only when three conditions hold: the current transaction is one

of a pre-declared set of injection indices, the controller is in the early cycles of the verify read window, and the pulse-mode bus carries a verify-related encoding. Outside this window the readback line carries the unperturbed shadow value. Two complementary injection scenarios are applied to disjoint index sets:

- **Under-programmed injection** drives the readback one step *below* the expected weight. The controller sees a short-of-target condition, increments the per-cell retry counter, and re-enters PROGRAM with the short single-cycle retry pulse train. The branches exercised are: VERIFY_CHECK under, retry-counter increment, PROG retry, eventual successful verify (after the injection window expires). If the injection persists past the retry budget, the FSM falls into the erase branch.
- **Over-programmed injection** drives the readback one step *above* the expected weight. The controller takes the over-programmed branch, enters ERASE with the verify-initiated flag, completes the erase sub-FSM, and then either returns to PROGRAM (if the global retry counter is still in budget) or terminates. The branches exercised are: VERIFY_CHECK over, verify-initiated erase, full erase sub-FSM, ERASE_COMPLETE → PROGRAM, and a re-run of the program-verify sequence on the same cell.

Coverage discipline. Each program-verify regression run is constrained to fail unless both the under- and over-programmed paths have been observed at least once. This guards against accidental regressions in the injection logic or the gating window being silently absorbed by a still-passing scoreboard.

5.4 End-of-Run Checks

Three classes of check close the verification loop. *Static-contract checks* compare observed outputs against expected values computed by the same helper functions the RTL uses (so the comparison is a true contract check rather than a re-implementation). The *shadow-matrix scoreboard* compares post-run shadow contents against the expected weight vector; because every successful verify writes the expected value into the shadow, a clean scoreboard confirms that the recovery policy converged the matrix to the intended state for every cell. The *branch-exercise check* asserts that each recovery branch has been entered at least once, preventing the regression from silently degenerating into a happy-path check.

5.5 Confidence Argument for Analog Integration

No claim is made here about analog correctness. What *is* claimed is that the controller’s digital behavior - every state transition, every pulse train, every recovery branch, every transaction-word format - has been exercised under the same digital boundary contract that the analog core will eventually expose. The controller is verified *in situ* (together with the parallel interface and the input buffer, in real host-to-core order), *by construction* (fault injection forces every recovery branch to be visited), and *composably* (a higher-fidelity model can replace the mock without disturbing the surrounding regressions).

What remains for the analog phase is therefore confined to the analog domain: device variability, pulse-amplitude sensitivity, IR drop on the array, and timing of the *op_done* handshake relative to actual cell settling. None of those concerns invalidate the digital contract; they refine it. The functional regressions established here become the baseline that the analog phase reuses, and any divergence observed in mixed-signal simulation is then attributable to the new model rather than to a previously unverified piece of the FSM.

Chapter 6

Results

The simulation evidence validates the architectural intent of the design. The presentation is organized around the three claims of Section 5.1 - the static transaction-word contract, the dynamic FSM protocol, and the closed-loop recovery policy. The directed regression set is summarized quantitatively in Table 6.1, and the two most architecturally informative experiments - the full closed-loop program-verify-erase recovery and the inference dataflow - are illustrated in the waveform captures of Sections 6.4 and 6.5. Per-test waveform captures and console logs for every entry in Table 6.1 are archived in the project repository.

6.1 Regression Coverage and Outcomes

Table 6.1 summarizes every directed test in the regression set, grouped by the architectural claim it discharges and annotated with the number of stimulus cases it covers and the resulting pass/fail outcome. All 60+ directed cases pass with zero scoreboard mismatches. Note that “PI” in the table refers to the Parallel Interface.

Table 6.1: Directed regression coverage. Each row corresponds to a focused test of one architectural claim. *Cases* counts the number of distinct stimuli inside the bench.

Test	Architectural claim exercised	Cases	Outcome
Address & pulse-mode packing	Static contract: outbound core word matches the host-requested cell, and the pulse-mode bus carries the encoding appropriate to the command class. Randomized PE, SA, column, row, and payload byte.	30	PASS
PI / Controller / Buffer integration	Dynamic protocol of our modules stack under mixed command traffic, with idle gating between transactions (host reads, programs, erases, and a full inference burst).	10	PASS
Host read reorder	Stability of the captured-address register and address-to-core-word reconstruction under out-of-program access order. Eight cells, fixed reordered read schedule.	8	PASS
Host erase selectivity	Spatial selectivity of host-initiated erase: erased cell is cleared while neighbor at a distinct address is preserved.	2	PASS
Inference dataflow	Collect → compute → result flow, bit-serial output lanes, and bit-select advance from zero to seven. Sanity check: compute must not start before eight pixel writes.	1 (9 beats)	PASS

Continued on next page...

Table 6.1: Directed regression coverage (continued)

Test	Architectural claim exercised	Cases	Outcome
10-weight program-verify	Compact regression with all four per-cell recovery scenarios: match, under (retry), over (erase), retry-exhausted (fall to erase). Suitable for GUI waveform debug.	10	PASS
Full program-verify regression	Closed-loop recovery policy at scale, with deliberate under- and over-programmed fault injection at disjoint pre-declared index sets spanning the whole run. End-of-run shadow-matrix scoreboard + branch-exercise discipline.	≥ 640	PASS
Unit benches (buffer $\times 3$, PI)	Per-module contracts in isolation: one-hot validity, reset-to-zero buffer, full-capacity overwrite, bit-serial output.	4	PASS

The remainder of the chapter interprets these outcomes against the architectural claims.

6.2 Static Contract: Address and Pulse-Mode Packing

The 30-case address-and-pulse-mode regression sweeps randomized cell coordinates and payload bytes through program, read, and erase commands. For each case the testbench reconstructs the expected outbound core word and pulse-mode encoding from the same helper functions used by the RTL, and compares them against the live outputs bit-for-bit throughout the active phase. Every case passes. This validates two things at once: the bidirectional addressing round-trip (host one-hot tail $\rightarrow$ packed binary $\rightarrow$ rebuilt one-hot tail) is implemented consistently across the interface and the controller, and the command-to-pulse-mode mapping is correctly wired through the pulse-output combinational block. Static-contract regressions are the floor on which every dynamic claim subsequently rests; their cleanliness is what makes FSM-level conclusions interpretable.

6.3 Dynamic Protocol: All Modules Integration

The integration regression exercises all implemented modules with ten labelled scenarios that span host reads, host programs, host erases, and inference, with idle gating between transactions. Each program command leaves the addressed buffer slot updated with the host-supplied payload, reflecting the data-load write that the program sub-FSM performs in its **SELECT** state. Read and erase commands leave the buffer untouched. The nine-beat inference burst loads exactly eight pixels into the buffer before transitioning to compute, and the bit-serial lanes expose the expected per-bit pattern during the compute window.

What this regression establishes is the consistency of the dynamic protocol *across* module boundaries: there is no missed handshake between the interface and the controller, no contention between the controller’s buffer-mode selection and the buffer’s mode-qualified ports, and no state-machine residue from one transaction leaking into the next. The host read reorder, host erase selectivity, and unit benches further narrow this claim down to specific design risks (stale captured-address register, mis-routed erase decode, per-module contracts) and all return clean. With the static and dynamic regressions both clean, the closed-loop recovery claim is the only remaining architectural claim that needs separate evidence.

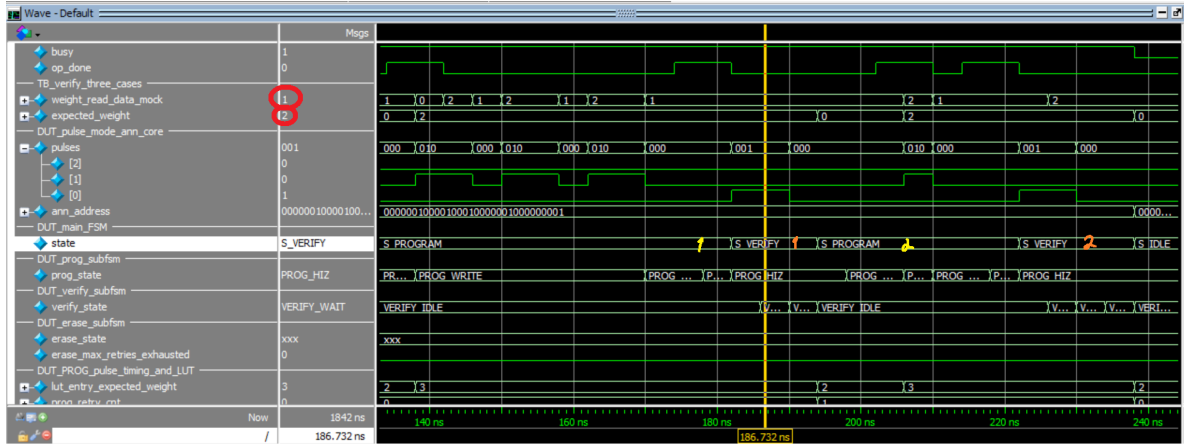


Figure 6.2: Closed-loop recovery (Under-programmed case): A readback mismatch (read 1 vs. expected 2) triggers the retry logic. The FSM immediately loops back into S_PROGRAM to apply the short single-cycle nudge train without returning to idle.

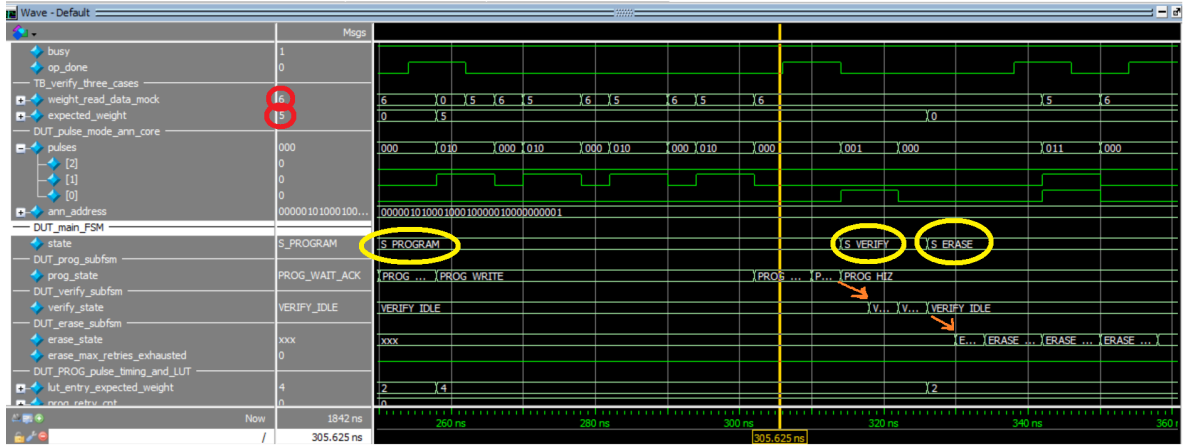


Figure 6.3: Closed-loop recovery (Over-programmed case): The FSM detects a severe overshoot (read 6 vs. expected 5) in S_VERIFY. This forces the controller into the penalty S_ERASE phase to clear the cell, before automatically returning to S_PROGRAM to rewrite the data.

Asymmetric injection. The under-programmed set is deliberately larger than the over-programmed set, reflecting the device-physics observation that under-programming is the more common analog failure mode - a single program pulse undershoots more often than it overshoots. The regression therefore spends more time exercising the retry path than the erase path, in proportion to the expected real-world distribution.

6.5 Inference Dataflow

The inference regression validates the bit-serial output discipline of the input buffer and the cycle-by-cycle advance of the bit-select line in the controller - the two pieces of the design that the analog MVM array depends on for its per-bit-time input vector. A single misalignment between the bit-select advance and the buffer's lane outputs would corrupt the entire MVM result, so this regression is the structural analogue of the recovery for the read-only path.

A nine-beat inference burst drives the controller through collect, compute, and result. Eight pixel writes are observed in the buffer before any compute indicator fires, the bit-select

counter steps from zero to seven during compute, and the eight bit-serial lanes present the selected bit of each of the eight buffered pixels in parallel. Figures 6.4 to 6.6 capture this sequence from the macroscopic FSM level down to the cycle-accurate bit lanes. Segmenting the transaction clearly proves the alignment between the *bit_sel* line and the parallel lane outputs-the critical invariant on which the analog MVM result depends.

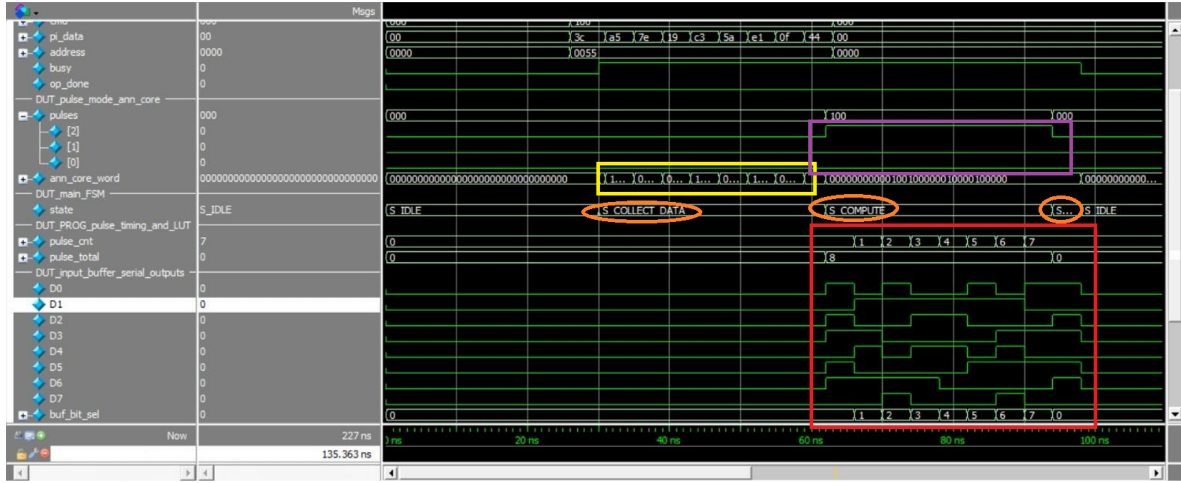


Figure 6.4: Inference dataflow (Transaction overview): The waveform shows the FSM progressing linearly through $S_COLLECT_DATA \rightarrow S_COMPUTE \rightarrow S_IDLE$. **Annotations:** *Yellow oval:* FSM state transitions. *Yellow rectangle:* `ann_address` during the data collection phase. *Purple rectangle:* Asserted `pulses` bus during the compute phase. *Red rectangle:* Bit-serial outputs (`D0-D7`) and the `buf_bit_sel` counter incrementing from 0 to 7.

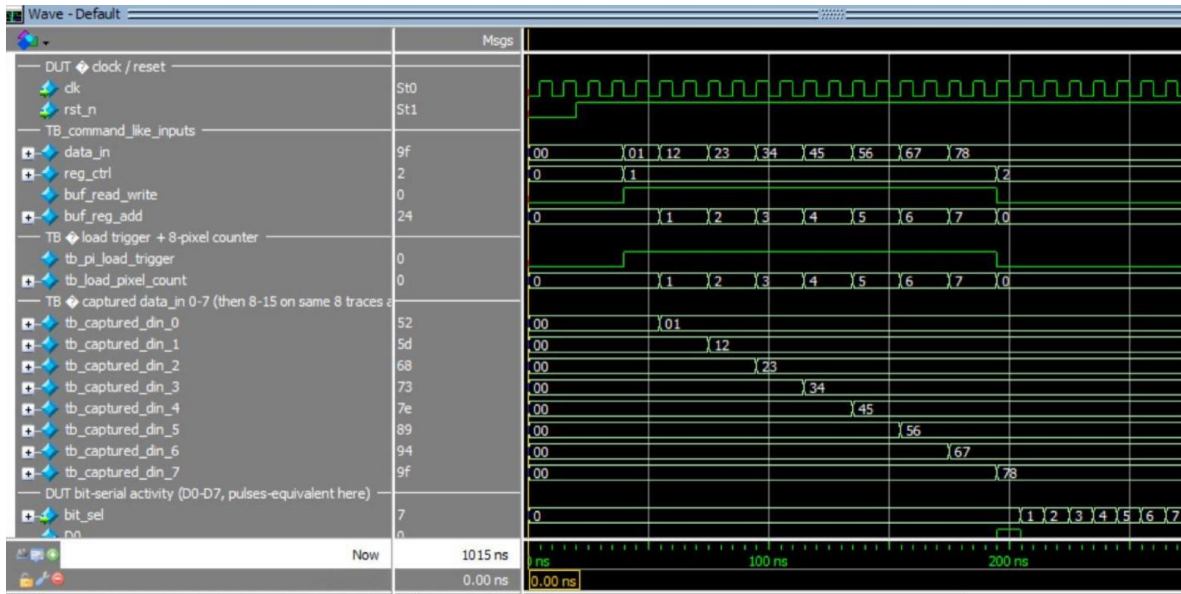


Figure 6.5: Inference dataflow (Collect phase detail): Eight successive data-load cycles capture the incoming pixels into the buffer registers, driven by the incrementing write address.

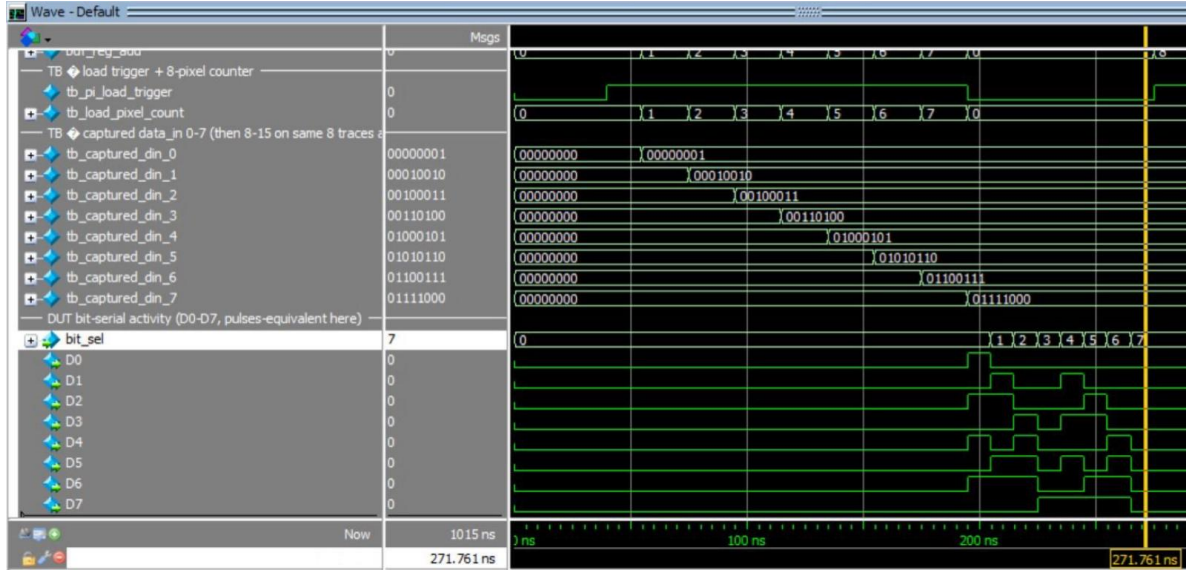


Figure 6.6: Inference dataflow (Compute phase detail): The *bit_sel* line steps from 0 to 7 across consecutive cycles. Synchronously, the eight bit-serial lanes (D0–D7) extract and present the corresponding bit of each captured pixel in parallel.

6.6 What the Results Establish and What They Do Not

The collected evidence establishes, under explicit modeling assumptions, that the controller’s digital behavior conforms to its architectural intent across every claim of Section 5.1. The static contract is honored on every directed case; the dynamic protocol is honored on every integration scenario; the closed-loop recovery policy converges every weight in the full regression to its expected value in the presence of deliberately injected failures; and the inference dataflow correctly stages pixels and presents the expected per-bit values during compute.

Equally important to be explicit about what is *not* established. Timing closure, physical feasibility, and gate-level power/area are out of scope - no synthesis or place-and-route is in the verification flow. End-to-end numerical correctness of an analog MVM is not established either, because no golden MAC reference exists in the environment. And the controller’s behavior under analog non-idealities not captured by the four-bit quantized shadow matrix - in particular, the timing of the *op_done* handshake relative to actual cell settling, and the cell-level variability of the Y-Flash device under repeated programming - remains a question for the analog phase, not for the digital phase reported here. These scope limits motivate the follow-on roadmap of Chapter 7.

Chapter 7

Limitations and Future Work

Finally, it is necessary to catalog the limitations of the present work and proposes a concrete roadmap for follow-on effort, grouped by their architectural source: the boundary between the digital controller and the analog core, the structural residue of the FSM design, and the verification scope.

7.1 Limitations and Technical Debt

Behavioral mock vs. analog reality. The behavioral mock of Chapter 5 captures the digital contract at the boundary - shadow weight matrix, readback line, *op_done* handshake - but discards every analog observable. The scope of what is validated is therefore explicitly *control sequencing* and *digital packet semantics* against a first-order behavioral abstraction, not full accelerator accuracy [1, 3]. Three specific caveats follow. The *physical timing of op_done* is testbench-generated and does not yet have a silicon-characterized tolerance band against charge-pump settling and in-cell transients. The *end-to-end MVM numerical accuracy* cannot be claimed because the environment has no golden MAC reference: the shadow matrix captures the per-cell quantized weight, not the analog MVM result. The *analog non-idealities* of the array - inter-cell cross-talk, IR drop along long lines, retention drift between programming and verify, and device-to-device variability beyond the discrete four-bit readback injection patterns - are not modeled.

Verification scope. The verification environment is directed. There is no UVM infrastructure, no constrained-random stimulus engine, no functional coverage model, and no formal property set. The absence of these techniques is a scope decision, not a feasibility claim, and their inclusion is the natural next step (Sections 7.2 and 7.3).

7.2 Future Work - Constrained-Random Stimulus and Coverage

The existing environment already exposes a legal-packet generator: the helper functions that build a transaction word from indices, and the one-hot validity predicates, together form a constructive way to generate and filter random host packets. A natural extension is a constrained-random environment with four pieces. First, randomize host data and command subject to one-hot tail validity and the enumerated command set. Second, constrain to the protocol: the controller does not support pipelined host issue, so the random stream must idle until busy drops, mirroring the directed bench discipline. Third, define covergroups on the main state, the three sub-FSM states, and the three verify-check branches, together with crosses of command class at idle. Fourth, reuse the shadow-matrix scoreboard from the existing program-verify regression to self-check each randomly generated transaction.

7.3 Future Work - Synthesis, Integration, and Post-Silicon Validation

Gate-level synthesis. Once the RTL and functional verification stabilize, logic synthesis, static timing analysis (STA), and power/area characterization are the natural next steps. This phase requires mapping the design to a specific target technology library, defining clock and reset constraints, and generating a gate-level netlist. The functional regressions established in this project remain valid post-synthesis and can be reused as a gate-level confidence baseline.

Mixed-signal integration. The digital controller must ultimately be integrated with the physical analog Y-Flash ANN core. Future work includes mixed-signal co-simulation to validate the true analog timing of the *op_done* handshake, charge-pump settling latencies, and in-cell programming transients. This integration will replace the behavioral mock with a high-fidelity analog model, ensuring the digital packet semantics hold under realistic electrical conditions.

Lab testing and silicon validation. Following a tape-out, post-silicon validation in the laboratory is critical. Physical testing will expose the controller to real-world analog non-idealities that are not fully captured in simulation. Key phenomena to characterize include On-Chip Variation (OCV), IR drop along long array lines, inter-cell cross-talk, and retention drift. Observing the closed-loop recovery policy in silicon will demonstrate its robustness against true device-to-device variability and validate the overall inference accuracy of the accelerator.

Chapter 8

Conclusion

This report has presented the RTL design and functional verification of a controller for an compute-in-memory Artificial Neural Network accelerator. The design comprises three synthesizable modules - a combinational host interface, a hierarchical FSM controller with three cooperating sub-FSMs for programming, verification, and erase, and a synchronous input buffer - arranged around a single transaction-word format and a multi-bit pulse-mode bus that drives a Y-Flash floating-gate memristor array.

The architectural contribution of the work is a closed-loop program-verify-(re-program or erase) recovery policy implemented as an FSM hierarchy: a single host-issued programming command is translated by the controller into an iterative sequence of analog pulses and verify reads, with re-programming on under-programmed readbacks within a configurable retry budget and a fall-back to erase on over-programmed readbacks or after retry exhaustion. The controller absorbs device-characterization data through a Look-Up Table that shapes the first-attempt programming pulse on a per-weight basis, with a short single-cycle nudge train reserved for subsequent corrections.

The methodological contribution of the work is a verification strategy that establishes the digital correctness of this recovery policy without access to the physical analog core. A behavioral mock of the ANN array exposes the same digital boundary contract that the real core will eventually present - a shadow weight matrix, an analog-side readback line, and an *op_done* handshake - and deliberate fault injection during the verify read window forces the controller into every branch of the recovery diagram. The full regression converges every weight in the matrix to its expected value in the presence of injected failures, while the targeted directed tests confirm the static transaction-word contract, the dynamic FSM protocol of the three modules we designed, the spatial selectivity of the erase path, and the bit-serial inference dataflow. Together, these results establish that the controller’s digital behavior conforms to its architectural intent across every claim of the verification plan.

The boundaries of the work are equally explicit. Behavioral mocks stand in for the physical memristor array; *op_done* and the readback line are idealized as deterministic functions of the FSM state; a handful of architectural hooks in the RTL (the currently-unreachable reset state, the two unused module parameters, the latent status signals) are deliberately left as integration points for a follow-on revision; and no synthesis, timing closure, or power/area artifacts are produced. A rigorous next phase is therefore not simply “more tests”, but a graduated extension of the verification: SystemVerilog assertions on the protocol boundaries, constrained-random stimulus with quantitative coverage of the recovery branches, and - when the device team can supply it - a higher-fidelity model of the Y-Flash cell that replaces the behavioral mock without disturbing the surrounding regression infrastructure [1, 3, 4].

The composability of the present environment is what makes this progression realistic. The mock-based regression has been designed so that every directed test that passes today will continue to run against a higher-fidelity model tomorrow, and the recovery-branch coverage established today carries forward into any future mixed-signal phase. The controller delivered here is therefore not merely a snapshot of digital behavior at one point in time; it is the defensible digital baseline against which the analog integration of the project’s broader Y-Flash research effort can be carried out.

Supplementary Material

Comprehensive technical documentation, including full module port tables, exhaustive FSM transition diagrams, complete testbench methodologies, and RTL source code, is available in the project's GitHub repository at <https://github.com/muhammad3mar/ann-accelerator-controller.git>

References

- [1] Daniele Ielmini and H.-S. Philip Wong. “In-memory computing with resistive switching devices”. In: *Nature Electronics* 1.6 (2018), pp. 333–343. DOI: [10.1038/s41928-018-0092-2](https://doi.org/10.1038/s41928-018-0092-2).
- [2] Naveen Verma, Hongyang Jia, Hossein Valavi, Yinqi Tang, Murat Ozatay, Lung-Yen Chen, Bonan Zhang, and Peter Deaville. “In-Memory Computing: Advances and Prospects”. In: *IEEE Solid-State Circuits Magazine* 11.3 (2019), pp. 43–55.
- [3] Abu Sebastian, Manuel Le Gallo, Riduan Khaddam-Aljameh, and Evangelos Eleftheriou. “Memory devices and applications for in-memory computing”. In: *Nature Nanotechnology* 15.7 (2020). IBM Research – Zurich, pp. 529–544.
- [4] Wei Wang, Loai Danial, Eric Herbelin, Barak Hoffer, Batel Oved, Tzofnat Greenberg-Toledo, Evgeny Pikhay, Yakov Roizin, and Shahar Kvatinsky. “Physical based compact model of Y-Flash memristor for neuromorphic computation”. In: *arXiv preprint arXiv:2202.10228* (Feb. 2022). The Andrew and Erna Viterbi Faculty of Electrical and Computer Engineering, Technion—Israel Institute of Technology, and Tower Semiconductor. URL: <https://arxiv.org/abs/2202.10228>.